Rapport de stage de fin de formation
Présenté en vue de l'obtention du diplôme de :

TECHNICIEN SPÉCIALISÉ
EN DÉVELOPPEMENT DIGITAL

Sous le thème

Développement et conception d'une application web de gestion des finances personnelles pour l'entreprise GEBS

Réalisé par

Mme BIRAM Nada

Organisme : **SOCIETE GEBS** Logo :

Encadré par : **Mr. ZERKEF ABDELLAH** Encadrant professionnel
Encadré par : **Mme. GUELSA MOUNA** Encadrant pédagogique

Soutenu le 06/07/2026. Devant le jury :

Mme. OURRAHTE MALAK	Formateur en Développement Digital	***Membre du jury***
Mme. FRITET IMANE	Formateur en Développement Digital	***Membre du jury***

Dédicaces

Je dédie ce travail avec une profonde gratitude à :

Ma chère famille,
Pour son amour, son soutien indéfectible, ses sacrifices et ses encouragements tout au long de mon parcours. Leur confiance, leur patience et leur présence m'ont permis d'avancer avec détermination et de mener à bien cette étape importante de ma formation.

Mes formateurs,
Pour la qualité de leur enseignement, leurs conseils, leur disponibilité et les connaissances qu'ils m'ont transmises tout au long de ma formation. Leur accompagnement a grandement contribué à mon évolution tant sur le plan académique que professionnel.

Mes amis et mes proches,
Pour leur soutien moral, leurs encouragements et les nombreux moments de partage qui ont rendu ce parcours plus agréable. Leur présence et leur confiance ont été une source de motivation tout au long de cette expérience.

À toutes les personnes qui, de près ou de loin, ont contribué à mon apprentissage et à la réussite de ce stage, j'adresse mes plus sincères remerciements.

Remerciements

Au terme de ce stage de fin d'études, je tiens tout d'abord à adresser mes sincères remerciements à la **CMC-Cité des Metiers et Compétences** pour la qualité de la formation dispensée, qui m'a permis d'acquérir les connaissances et les compétences nécessaires à la réalisation de ce projet. J'exprime également ma profonde gratitude à l'entreprise **GEBS – Global Electronic Business Solutions** pour m'avoir accueilli au sein de son équipe et m'avoir offert l'opportunité d'effectuer ce stage dans un cadre professionnel enrichissant.

Cette expérience m'a permis de découvrir le monde du travail, de développer mes compétences techniques et d'acquérir de nouvelles connaissances dans le domaine des technologies de l'information et des solutions numériques.

Je remercie également l'ensemble des membres de l'entreprise pour leur accueil chaleureux, leur disponibilité, leur patience ainsi que les conseils qu'ils m'ont apportés tout au long de cette période. Leur accompagnement et les échanges professionnels que j'ai pu avoir avec eux ont beaucoup contribué à mon apprentissage et à mon intégration au sein de l'entreprise.

Mes sincères remerciements s'adressent aussi à mon encadrant de stage pour son suivi, ses orientations et son soutien durant la réalisation des différentes tâches qui m'ont été confiées.

Je remercie également mes formateurs et encadrants pédagogiques pour les connaissances acquises durant ma formation ainsi que pour leurs conseils et leur accompagnement.

Enfin, j'adresse toute ma gratitude à ma famille, mes proches et mes amis pour leur soutien moral, leurs encouragements et leur confiance tout au long de cette expérience.

Table des matières

Dédicaces **1**

Remerciements **2**

Acronymes **8**

Introduction Générale **10**

Chapitre 1 : Contexte Général du Projet **11**

1.1 Introduction . . . 12
1.2 Présentation de l'Entreprise d'Accueil . . . 12
1.3 Cadre du projet . . . 13
1.3.1 Problématique . . . 13
1.3.2 Analyse de l'existant . . . 13
1.3.3 Solution proposée . . . 13
1.3.4 Maquettes . . . 14
1.4 Analyse et spécification des besoins . . . 14
1.4.1 Besoins fonctionnels . . . 14
1.4.2 Besoins non fonctionnels . . . 15
1.5 Gestion du projet . . . 15
1.6 Planification prévisionnelle . . . 16
1.7 Conclusion . . . 16

Chapitre 2 : Analyse et Conception **18**

2.1 Introduction . . . 19
2.2 Méthodes et outils de modélisation . . . 19
2.2.1 Langage de modélisation UML . . . 19
2.2.2 Le processus unifié . . . 19
2.2.3 Prototypage des interfaces utilisateur . . . 19

2.2.4 Identification des acteurs et des cas d'utilisation . . . 22
2.3 Diagramme de cas d'utilisation . . . 23
2.4 Étude des diagrammes de séquence . . . 24
2.4.1 Diagramme de séquence : Ajout d'une dépense . . . 25
2.4.2 Diagramme de séquence : Virement entre comptes . . . 26
2.5 Diagramme de classes . . . 26
2.6 Conclusion . . . 28

Chapitre 3 : Réalisation Technique 29
3.1 Introduction . . . 30
3.2 Architecture générale du projet . . . 30
3.3 Outils, langages et technologies utilisés . . . 31
3.3.1 React.js . . . 31
3.3.2 Laravel . . . 32
3.3.3 Tailwind CSS . . . 32
3.3.4 Axios . . . 32
3.3.5 Laravel Sanctum . . . 32
3.3.6 Lucide React . . . 32
3.3.7 MySQL . . . 33
3.3.8 Postman . . . 33
3.3.9 Laravel Telescope . . . 33
3.3.10 TipTap . . . 33
3.3.11 Git . . . 34
3.3.12 Figma . . . 34
3.3.13 Visual Studio Code . . . 34
3.4 Gestion de l'authentification et de la sécurité . . . 34
3.5 Communication entre le frontend et le backend . . . 35
3.6 Conclusion . . . 35

Chapitre 4 : Réalisation et Démonstration de l'Application 36
4.1 Introduction . . . 37
4.2 Landing Page . . . 37
4.2.1 En-tête et présentation de l'application . . . 37
4.2.2 Présentation des fonctionnalités . . . 38
4.2.3 Présentation du journal Nikki . . . 39
4.2.4 Formulaire de contact . . . 39
4.2.5 Appel à l'action final . . . 40

4.3 Authentification 40
4.4 Tableau de bord 41
4.5 Gestion des comptes 43
4.6 Gestion des transactions 44
4.6.1 Revenus 44
4.6.2 Dépenses 44
4.6.3 Virements 45
4.7 Ledger (Grand livre) 46
4.8 Budgets 47
4.9 Statistiques 48
4.10 IA financière 50
4.11 Notes et Checklists 51
4.12 Journal Nikki 52
4.12.1 Accueil 52
4.12.2 Vue Livre et Effet Page-Flip 52
4.12.3 Création d'une entrée 53
4.13 Profil utilisateur 55
4.14 Paramètres et Préférences 55
4.15 Administration 56
4.16 Conclusion 58

Conclusion Générale **59**

Table des figures

1.1 Diagramme de Gantt 16
2.2 Maquette Figma du tableau de bord de l'application 20
2.3 Maquette Figma du journal personnel Nikki 21
2.4 Maquette Figma des modales de transaction 22
2.5 Diagramme de cas d'utilisation général du système 24
2.6 Diagramme de séquence : processus d'ajout d'une dépense 25
2.7 Diagramme de séquence : processus de virement entre comptes 26
2.8 Diagramme de classes structurel du système 28
3.9 Architecture générale de l'application (Client-Serveur) 31
4.10 Logo de l'application MoneyIO 37
4.11 En-tête de la Landing Page 38
4.12 Présentation des fonctionnalités principales 38
4.13 Présentation du journal Nikki 39
4.14 Formulaire de contact 40
4.15 Appel à l'action final et pied de page 40
4.16 Interface d'authentification (Connexion / Inscription) 41
4.17 Interface principale du Tableau de bord 42
4.18 Widget de gestion des comptes (All Accounts) 43
4.19 Modal d'ajout d'un revenu 44
4.20 Modal d'ajout d'une dépense 45
4.21 Modal de virement entre comptes 46
4.22 Grand livre des transactions 47
4.23 Alerte de dépassement de budget 48
4.24 Page Statistiques 49
4.25 Module AI Financial Insights 50
4.26 Module de gestion des Notes et Checklists 51
4.27 Écran d'accueil du journal financier Nikki 52
4.28 Consultation double-page et effet d'animation page-flip 53

4.29 Interface de création et d'édition d'une entrée Nikki 54
4.30 Page de gestion du Profil Utilisateur 55
4.31 Panneau des Préférences et Paramètres globaux 56
4.32 Panneau d'administration 57

Acronymes

API	Application Programming Interface
CMS	Content Management System
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DB	Database
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IA	Intelligence Artificielle
IDE	Integrated Development Environment
i18n	Internationalization (Internationalisation)
JS	JavaScript
JSON	JavaScript Object Notation
MVC	Model View Controller
MVP	Minimum Viable Product
ORM	Object Relational Mapping
PHP	PHP Hypertext Preprocessor
REST	Representational State Transfer
SPA	Single Page Application

SQL	Structured Query Language
MySQL	My Structured Query Language
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UX	User Experience
VS Code	Visual Studio Code

Introduction Générale

La gestion financière personnelle représente aujourd'hui un enjeu quotidien pour un grand nombre de personnes. Entre la multiplication des comptes bancaires, la diversité des sources de revenus et la complexité croissante des dépenses, il devient difficile de maintenir une vision claire et organisée de sa situation financière sans disposer d'outils adaptés.

C'est dans ce contexte que s'inscrit le présent projet de fin de formation, réalisé dans le cadre d'un stage au sein de l'entreprise. L'objectif principal était de concevoir et développer une application web complète permettant à l'utilisateur de centraliser la gestion de ses finances personnelles, de suivre ses transactions, de planifier ses budgets et d'organiser ses données personnelles au sein d'une interface moderne et intuitive.

L'application développée, baptisée **MoneyIO**, repose sur une architecture full-stack moderne combinant React pour le frontend et Laravel pour le backend, avec une communication sécurisée via une API REST. Elle intègre plusieurs modules fonctionnels couvrant la gestion des comptes, des revenus et dépenses, des virements, des budgets, des notes ainsi qu'un journal personnel.

Afin de présenter de manière structurée le travail réalisé, ce rapport est organisé en quatre chapitres :

- Le **premier chapitre** présente le contexte général du projet, l'organisme d'accueil, la problématique identifiée ainsi que la solution proposée.
- Le **deuxième chapitre** est consacré à l'analyse et à la conception du système, à travers les diagrammes UML et les maquettes réalisées sur Figma.
- Le **troisième chapitre** détaille les choix techniques effectués, les technologies utilisées ainsi que l'architecture et la réalisation de l'application.
- Le **quatrième chapitre** présente une démonstration des différentes interfaces et fonctionnalités implémentées au sein de l'application.

Ce rapport se conclut par une conclusion générale faisant le bilan du travail accompli et proposant des perspectives d'évolution pour le projet.

Chapitre 1

Contexte Général du Projet

1.1 Introduction

Ce chapitre a pour objectif de présenter le cadre général dans lequel s'inscrit ce projet de fin de formation. Il introduit dans un premier temps l'organisme d'accueil au sein duquel le stage a été effectué, avant de définir la problématique ayant motivé le développement de l'application. La solution proposée ainsi que les besoins fonctionnels et non fonctionnels du système y sont également détaillés. Enfin, l'organisation et la planification du projet sont présentées afin d'illustrer la démarche adoptée tout au long de la réalisation.

1.2 Présentation de l'Entreprise d'Accueil

GEBS – Global Electronic Business Solutions est une entreprise marocaine spécialisée dans les technologies de l'information et les solutions numériques pour les entreprises. Fondée en 2010 par des ingénieurs ayant acquis une expérience dans le domaine des NTIC, l'entreprise accompagne ses clients dans leur transformation digitale à travers des solutions adaptées à leurs besoins.

GEBS intervient dans plusieurs domaines d'activité, notamment le développement informatique, les solutions de gestion, la commercialisation de matériels informatiques, la géolocalisation de véhicules ainsi que le consulting et la formation professionnelle.

L'entreprise développe également des logiciels de gestion destinés principalement au domaine pharmaceutique, en mettant l'accent sur la simplicité, la fiabilité, l'efficacité et la maîtrise des processus métiers. Elle propose aussi des solutions de géolocalisation permettant aux entreprises d'assurer un meilleur suivi et une gestion plus efficace de leurs flottes de véhicules.

Par ailleurs, GEBS commercialise différents équipements et matériels informatiques de marques reconnues et assure un accompagnement professionnel à ses clients grâce à son expertise technique et son engagement envers la qualité de service.

Grâce à son expérience, son professionnalisme et son intérêt pour l'innovation numérique, GEBS occupe une place importante dans le domaine des solutions digitales et de l'accompagnement des entreprises au Maroc.

1.3 Cadre du projet

1.3.1 Problématique

La gestion des finances personnelles est devenue une tâche importante dans le quotidien, notamment avec la multiplication des dépenses, des abonnements et des paiements numériques. Pourtant, beaucoup de personnes continuent à gérer leur budget de manière désorganisée, sans véritable outil leur permettant de suivre clairement leurs revenus, leurs dépenses ou l'évolution de leur situation financière.

Les applications existantes sont souvent soit trop complexes, soit limitées en fonctionnalités, ce qui rend leur utilisation peu pratique pour un usage quotidien. De plus, certaines solutions ne proposent pas une expérience utilisateur suffisamment intuitive ou personnalisable.

Dans ce contexte, il est apparu pertinent de développer une application permettant de centraliser la gestion budgétaire dans une interface moderne, simple à utiliser et accessible au quotidien.

L'objectif du projet est donc de concevoir une application web capable d'aider l'utilisateur à suivre ses transactions financières, organiser ses comptes et visualiser l'évolution de son budget de manière claire et interactive.

1.3.2 Analyse de l'existant

Actuellement, plusieurs solutions de gestion financière sont disponibles sur le marché. Cependant, beaucoup d'entre elles présentent certaines limites, notamment une interface surchargée, des fonctionnalités parfois payantes ou une expérience utilisateur peu adaptée aux besoins personnels.

Certaines applications se concentrent uniquement sur le suivi des dépenses sans offrir une vision globale du budget, tandis que d'autres manquent de flexibilité dans l'organisation des comptes, des catégories ou de l'historique des transactions.

Par ailleurs, il est fréquent que les utilisateurs utilisent plusieurs outils séparés pour gérer leurs notes personnelles, leurs rappels ou leurs objectifs financiers, ce qui complique davantage l'organisation quotidienne.

Cette analyse a mis en évidence la nécessité de proposer une solution centralisée, moderne et intuitive regroupant plusieurs fonctionnalités dans une seule application.

1.3.3 Solution proposée

Afin de répondre à cette problématique, une application web de gestion budgétaire nommée provisoirement **MoneyIO** a été développée.

L'application permet à l'utilisateur de :

— Gérer plusieurs comptes financiers ;
— Enregistrer les revenus, dépenses et transferts ;
— Consulter un tableau de bord interactif avec des statistiques et graphiques ;
— Suivre l'historique des transactions dans un grand livre (*Ledger*) ;
— Organiser des notes et checklists liées à la gestion personnelle ;
— Utiliser un journal financier permettant de suivre son évolution quotidienne.
— Générer des analyses financières personnalisées grâce à un module d'intelligence artificielle.

L'application repose sur une architecture moderne séparant le frontend et le backend afin d'assurer de bonnes performances, une meilleure maintenabilité et une expérience utilisateur fluide.

1.3.4 Maquettes

Avant la phase de développement, plusieurs maquettes ont été réalisées à l'aide de l'outil **Figma** afin de définir l'apparence générale de l'application ainsi que l'organisation des différentes interfaces.

Cette étape a permis de concevoir les principales pages de l'application, notamment le tableau de bord, le module *Ledger*, les notes et le journal financier. Les maquettes ont également servi à réfléchir à l'ergonomie, à la navigation et à l'expérience utilisateur avant l'implémentation technique.

1.4 Analyse et spécification des besoins

1.4.1 Besoins fonctionnels

L'application développée s'appuie sur plusieurs modules principaux permettant à l'utilisateur de gérer efficacement ses finances personnelles. Le système est organisé autour de différentes entités telles que les comptes, les transactions, les budgets, les notes et les utilisateurs.

Chaque utilisateur dispose de son propre espace de gestion lui permettant d'enregistrer et de consulter ses données financières de manière centralisée. Les transactions peuvent être classées selon différentes catégories afin de faciliter le suivi des dépenses et des revenus.

Les principaux besoins fonctionnels identifiés sont les suivants :
— Gérer les comptes financiers de l'utilisateur ;
— Enregistrer les revenus, dépenses et transferts ;
— Catégoriser les transactions financières ;
— Consulter l'historique des opérations effectuées ;
— Visualiser des statistiques et graphiques financiers ;
— Consulter des statistiques financières

— Générer des analyses via l'IA
— gérer un journal personnel (Nikki)
— Gérer des notes et checklists personnelles ;
— Suivre l'évolution du budget à travers un tableau de bord interactif ;
— Gérer l'authentification et les profils utilisateurs.

1.4.2 Besoins non fonctionnels

L'application repose sur une architecture moderne basée sur **React** pour le frontend et **Laravel** pour le backend, garantissant une séparation claire entre la logique métier et l'interface utilisateur.

Les exigences non fonctionnelles définies pour le projet sont les suivantes :

— Interface responsive compatible avec les ordinateurs, tablettes et smartphones ;
— Interface multilingue (Français / English)
— Navigation fluide et dynamique sans rechargement complet des pages ;
— Sécurisation des données et des opérations utilisateur ;
— Architecture modulaire facilitant la maintenance et l'évolution du projet ;
— Expérience utilisateur intuitive et ergonomique ;
— Optimisation des performances et rapidité d'exécution ;
— Organisation claire du code à travers des composants, services et routes dédiées.
— Sécurité avec Laravel Sanctum

1.5 Gestion du projet

Le projet a été réalisé de manière autonome tout en bénéficiant d'un suivi régulier avec l'encadrant pédagogique. Afin d'assurer une bonne organisation du travail et un suivi efficace des différentes tâches, l'outil **Trello** a été utilisé tout au long du développement.

Une approche progressive a été adoptée durant la réalisation du projet :

— Conception des maquettes et réflexion sur l'expérience utilisateur ;
— Mise en place de la structure backend et de la base de données ;
— Développement des fonctionnalités principales de l'API ;
— Intégration et développement des interfaces frontend ;
— Réalisation de tests fonctionnels et corrections des anomalies ;
— Validation progressive des différentes fonctionnalités développées.

1.6 Planification prévisionnelle

La réalisation du projet s'est déroulée selon plusieurs étapes principales :

— Mise en place du projet : configuration de l'environnement, initialisation des dépôts et modélisation de la base de données ;

— Authentification : mise en place de Laravel Sanctum et des pages de connexion et d'inscription ;

— Développement de l'API Backend : implémentation des contrôleurs CRUD pour l'ensemble des entités ;

— Développement du Dashboard : tableau de bord avec graphiques, cartes bancaires animées et résumé financier ;

— Développement du Grand livre (Ledger) : historique des transactions avec filtres et édition inline ;

— Développement des Notes & Checklists : système de notes inspiré de Google Keep ;

— Développement du Journal Nikki : éditeur de texte riche, vue livre animée et suivi de série ;

— Qualité & Finalisation : tests, optimisations, sécurité et rédaction de la documentation.

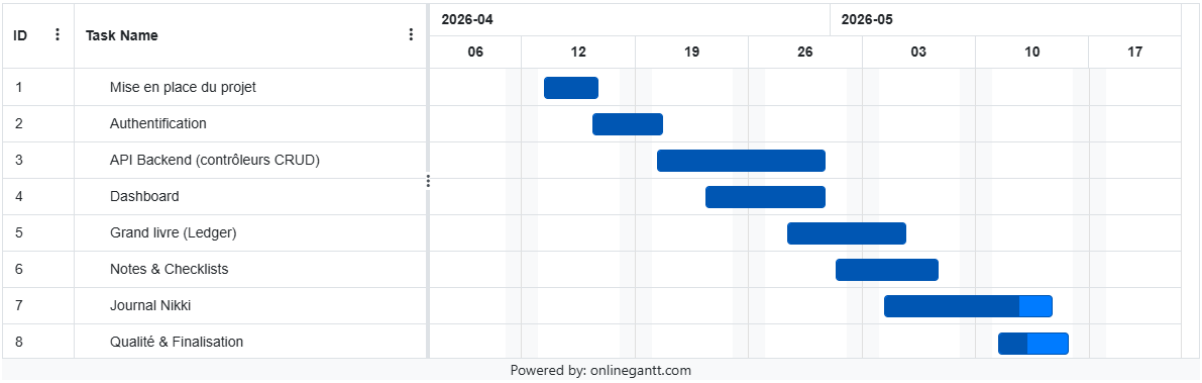


FIGURE 1.1 – Diagramme de Gantt

Ce diagramme de Gantt illustre la répartition temporelle des différentes phases de notre projet sur toute la période du stage. Il permet de visualiser le chevauchement des tâches, notamment la transition entre la conception des maquettes et le début des développements backend et frontend, garantissant ainsi le respect des délais fixés.

1.7 Conclusion

Ce chapitre a permis de présenter le contexte général du projet ainsi que les principales motivations ayant conduit au développement de cette application de gestion budgétaire. À travers

l'analyse de l'existant, les limites des solutions actuelles ont été mises en évidence, ce qui a permis de définir une solution adaptée aux besoins des utilisateurs.

Les besoins fonctionnels et non fonctionnels ont également été identifiés afin d'établir une base claire pour la conception et le développement de l'application. Enfin, l'organisation du projet ainsi que les différentes étapes de réalisation ont été présentées pour assurer un déroulement structuré du travail.

Le chapitre suivant sera consacré à l'analyse et à la conception du système, notamment la modélisation des fonctionnalités, l'architecture de l'application ainsi que les différents diagrammes UML utilisés durant le projet.

Chapitre 2

Analyse et Conception

2.1 Introduction

Avant d'entamer la phase de développement de l'application, une étape d'analyse et de conception a été réalisée afin de mieux définir les besoins fonctionnels et techniques du projet. Cette phase a permis d'identifier les différents acteurs du système, de structurer les fonctionnalités principales et de modéliser les interactions entre les différentes composantes de l'application.

Dans le cadre de ce projet, plusieurs outils de modélisation ont été utilisés afin de représenter le fonctionnement global du système ainsi que l'organisation des données et des interactions utilisateur.

2.2 Méthodes et outils de modélisation

2.2.1 Langage de modélisation UML

Le langage UML (Unified Modeling Language) est un standard largement utilisé dans la conception des systèmes informatiques. Il permet de représenter graphiquement les différents éléments d'une application ainsi que les interactions entre les utilisateurs et le système.

Dans ce projet, UML a été utilisé afin de modéliser les cas d'utilisation, les séquences d'interaction ainsi que les différentes entités principales de l'application.

2.2.2 Le processus unifié

Le processus unifié est une méthode de développement orientée objet reposant sur une approche itérative et incrémentale. Cette méthodologie permet d'organiser efficacement le développement du projet en plusieurs étapes tout en assurant une meilleure adaptation aux besoins fonctionnels du système.

L'approche adoptée durant ce projet a permis de structurer efficacement les différentes phases de conception et de développement de l'application.

2.2.3 Prototypage des interfaces utilisateur

Avant la phase de développement, plusieurs maquettes de l'application ont été réalisées à l'aide de l'outil Figma. Cette étape a permis de concevoir l'organisation des différentes interfaces ainsi que l'expérience utilisateur globale.

Les maquettes ont servi de référence durant le développement du frontend React et ont facilité la visualisation des différentes fonctionnalités de l'application, notamment le tableau de bord, la gestion des transactions, les budgets, les notes et le journal personnel. Certaines interfaces ont

toutefois été adaptées et enrichies durant la phase de réalisation afin d'améliorer l'expérience utilisateur et d'intégrer de nouvelles fonctionnalités.

Une attention particulière a été accordée à l'ergonomie, à la fluidité de navigation ainsi qu'à l'adaptabilité des interfaces sur différents supports (desktop, tablette et mobile).

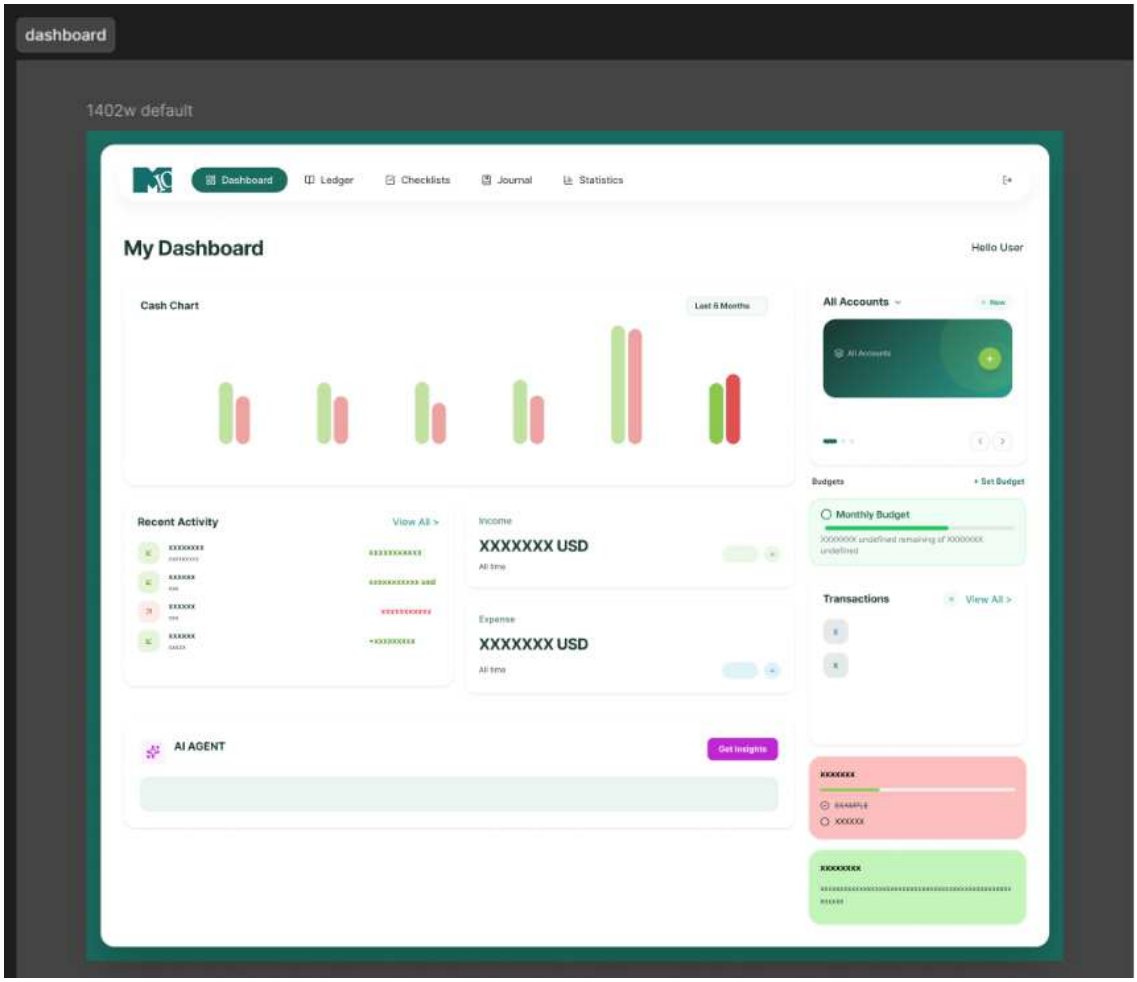


FIGURE 2.2 – Maquette Figma du tableau de bord de l'application

Cette maquette presente l'interface d'accueil de l'utilisateur, mettant en valeur de manière visuelle et interactive les statistiques budgétaires, le solde des comptes et les graphiques de répartition des dépenses.

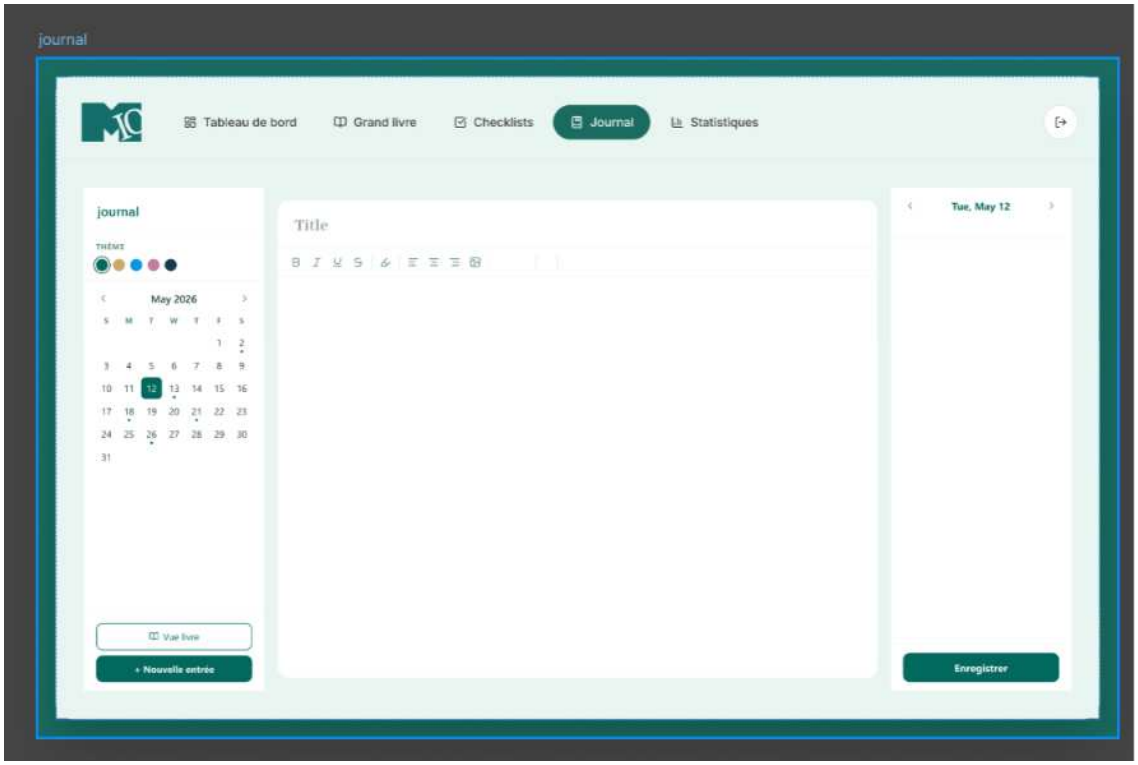


FIGURE 2.3 – Maquette Figma du journal personnel Nikki

Cette maquette illustre la conception du module Nikki, notamment l'organisation de l'éditeur de texte enrichi, l'intégration du système de mentions pour lier des transactions, ainsi que la disposition du calendrier et du résumé quotidien dans la barre latérale.

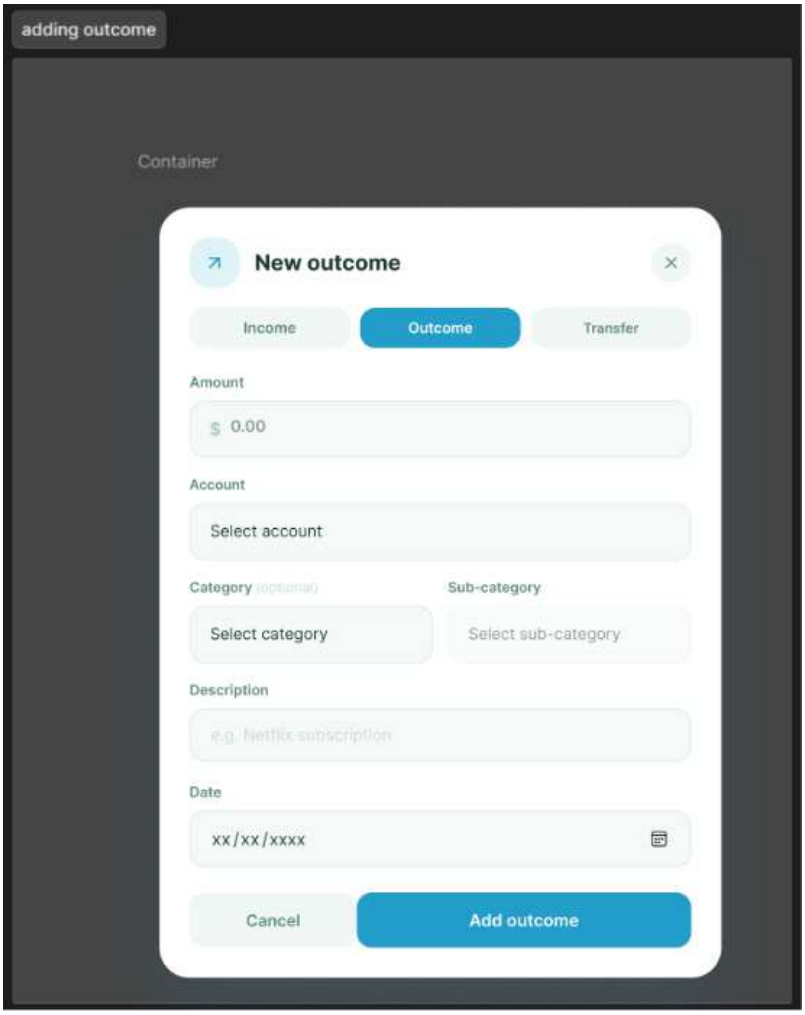


FIGURE 2.4 – Maquette Figma des modales de transaction

Avant leur développement, les formulaires d'ajout de revenus, de dépenses et de virements ont été prototypés sous la forme d'une fenêtre modale à onglets. Cette maquette a permis de définir les différents champs nécessaires à chaque type de transaction et de concevoir une interface de saisie simple, cohérente et intuitive..

2.2.4 Identification des acteurs et des cas d'utilisation

Dans le cadre de cette application de gestion budgétaire, deux acteurs principaux ont été identifiés :

— **Utilisateur :** Gère ses comptes, transactions, budgets et données personnelles ;

— **Administrateur :** Supervise les utilisateurs et assure la gestion globale de la plateforme.

Les principaux cas d'utilisation du système sont les suivants :

— Inscription et authentification des utilisateurs ;

— Gestion des revenus et des dépenses ;

— Réalisation de transferts entre comptes ;
— Création et suivi des budgets ;
— Consultation du tableau de bord financier ;
— Gestion des notes et checklists ;
— Rédaction d'entrées dans le journal personnel ;
— Supervision des données par l'administrateur.

2.3 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation ci-dessous illustre les différentes interactions possibles entre les acteurs du système et l'application de gestion budgétaire.

L'utilisateur peut accéder à son espace personnel afin de gérer ses comptes financiers, enregistrer des revenus et des dépenses, effectuer des virements, suivre ses budgets ainsi que consulter les statistiques disponibles dans le tableau de bord. L'application lui permet également de créer des notes, des checklists et des entrées de journal personnel.

L'administrateur dispose quant à lui de fonctionnalités supplémentaires lui permettant de superviser les utilisateurs, modifier les rôles, consulter les statistiques globales du système ainsi que gérer les paramètres généraux de l'application.

Ce diagramme met en évidence les principales fonctionnalités offertes par le système ainsi que les interactions entre les différents acteurs.

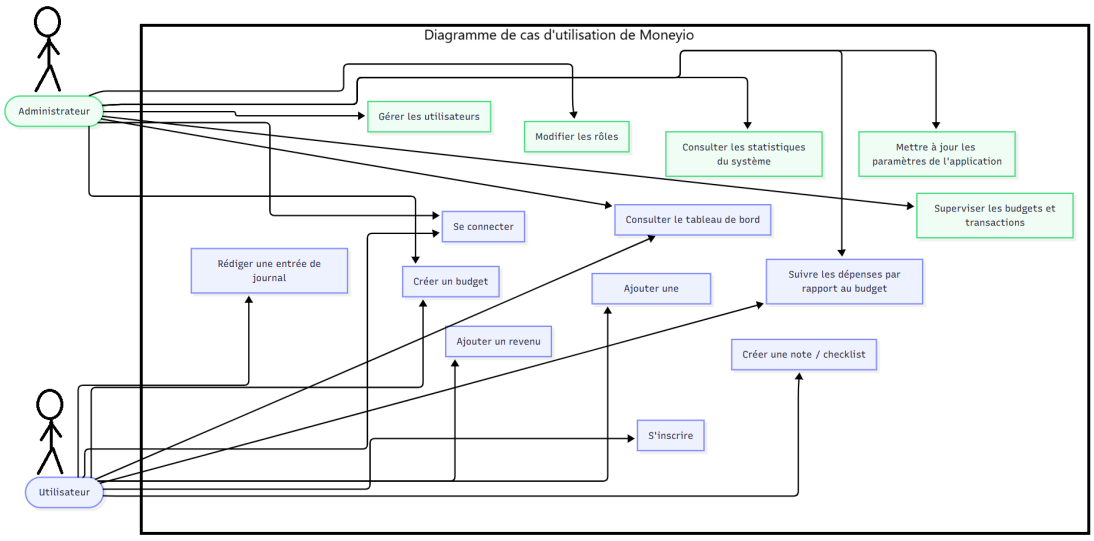


FIGURE 2.5 – Diagramme de cas d'utilisation général du système

Ce diagramme UML modélise les droits d'accès et les actions spécifiques attribuées à l'Utilisateur et à l'Administrateur, délimitant ainsi de manière exhaustive le périmètre fonctionnel de l'application MoneyIO.

2.4 Étude des diagrammes de séquence

Les diagrammes de séquence permettent de représenter chronologiquement les interactions entre les différents composants de l'application lors de l'exécution d'une fonctionnalité. Ils mettent en évidence les échanges entre l'utilisateur, le frontend React, les API Laravel, les contrôleurs ainsi que la base de données MySQL.

Dans le cadre de ce projet, deux scénarios représentatifs ont été modélisés. Le premier décrit le processus d'ajout d'une dépense, tandis que le second illustre le déroulement d'un virement entre deux comptes. Ces diagrammes montrent les différentes étapes de validation, les communications entre le frontend et le backend ainsi que le traitement des données avant leur enregistrement en base de données.

Cette modélisation permet de mieux comprendre le fonctionnement interne de l'application et met en évidence les mécanismes garantissant la sécurité, la cohérence et la fiabilité des opérations réalisées.

2.4.1 Diagramme de séquence : Ajout d'une dépense

Le diagramme suivant présente le déroulement du processus d'ajout d'une dépense. Après la saisie des informations par l'utilisateur, les données sont transmises au backend Laravel via une requête API sécurisée. Le contrôleur vérifie la validité des informations avant de les enregistrer dans la base de données. En cas de succès, le frontend est automatiquement mis à jour ; dans le cas contraire, un message d'erreur est retourné afin de permettre la correction des données.

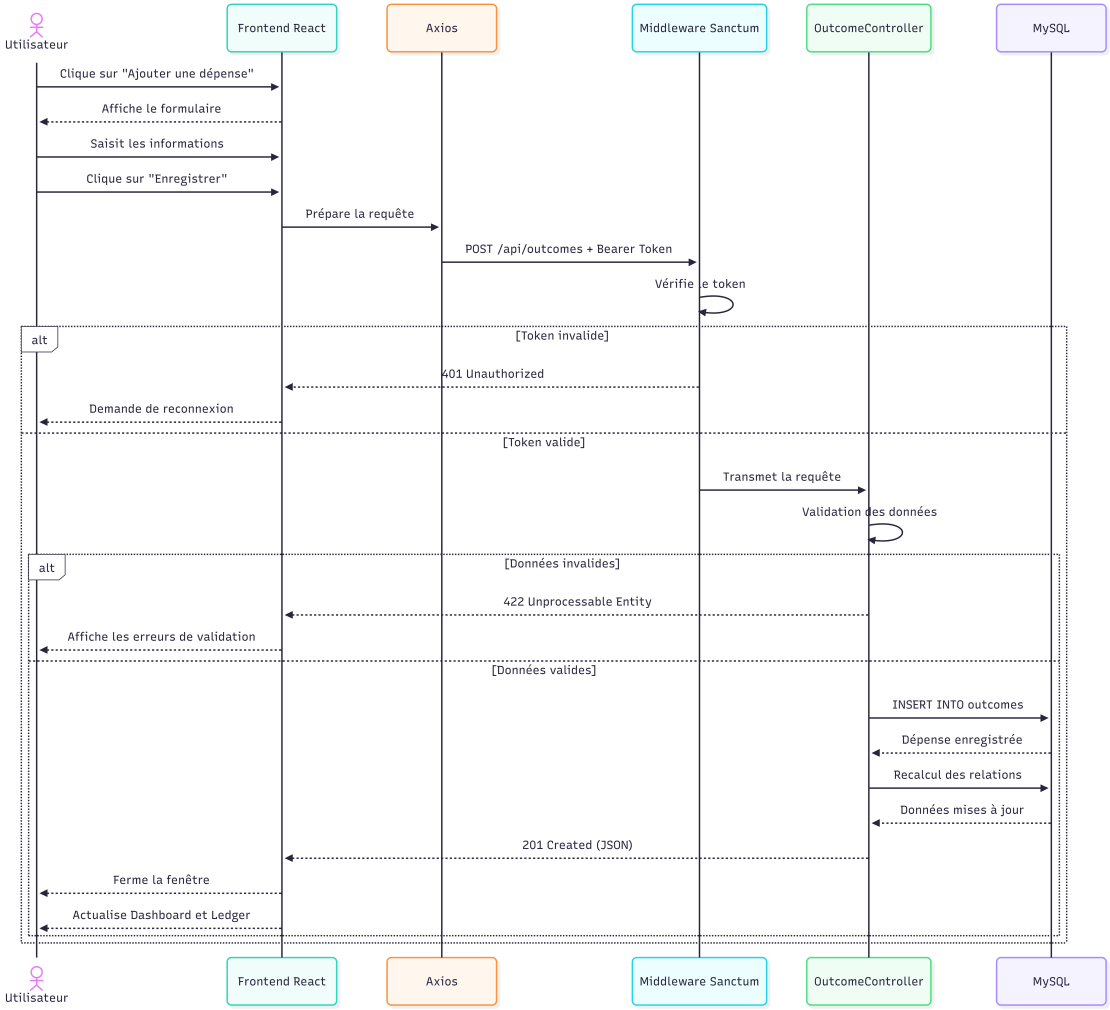


FIGURE 2.6 – Diagramme de séquence : processus d'ajout d'une dépense

Ce diagramme illustre les interactions entre React, Laravel Sanctum, le contrôleur des dépenses et la base de données MySQL lors de l'enregistrement d'une nouvelle dépense.

2.4.2 Diagramme de séquence : Virement entre comptes

Le diagramme ci-dessous représente le processus de virement entre deux comptes. Après la saisie des informations du virement, le backend vérifie la validité des données ainsi que le solde du compte source. Si les conditions sont satisfaites, le montant est débité du compte émetteur, crédité sur le compte destinataire puis le transfert est enregistré dans la base de données. Enfin, le frontend est mis à jour afin de refléter immédiatement les nouveaux soldes.

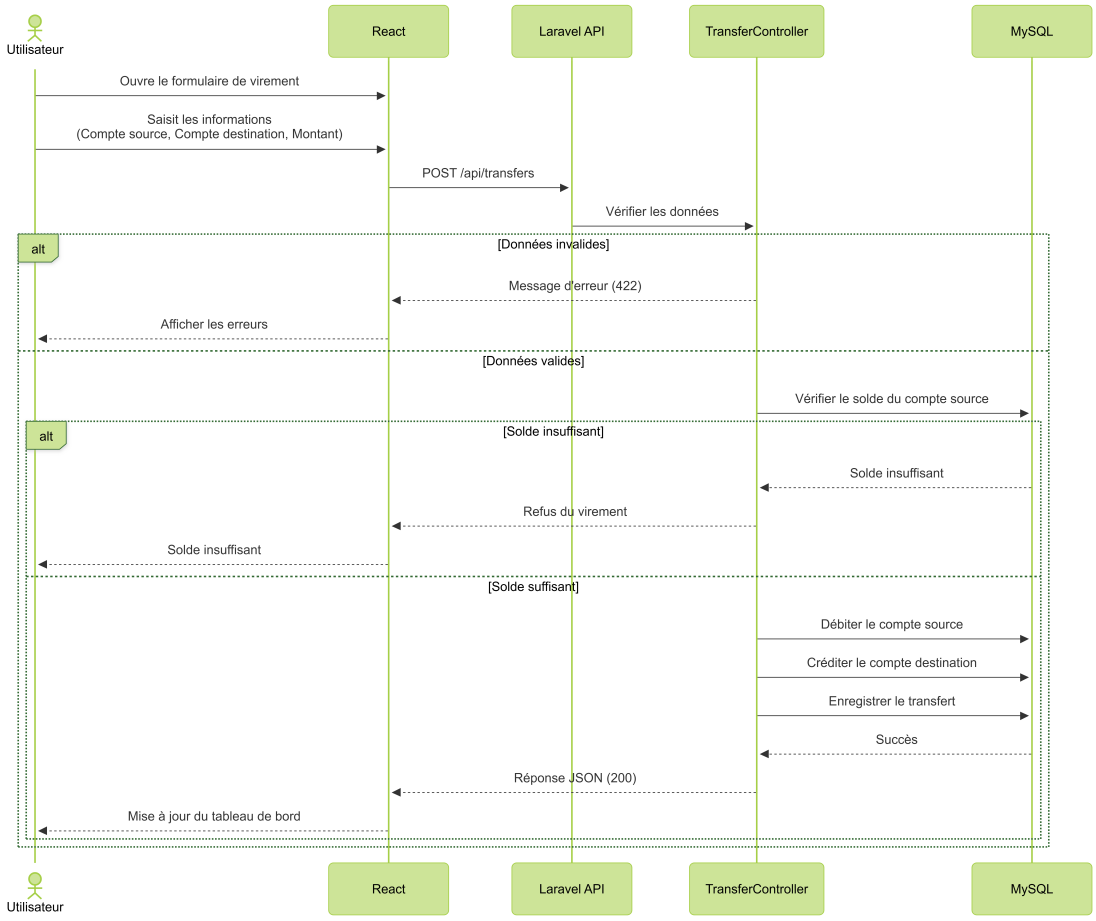


FIGURE 2.7 – Diagramme de séquence : processus de virement entre comptes

Ce diagramme présente les interactions entre l'utilisateur, le frontend React, les services Laravel et la base de données MySQL lors de l'exécution d'un virement entre deux comptes.

2.5 Diagramme de classes

Le diagramme de classes présenté illustre la structure principale de l'application ainsi que les relations entre les différentes entités du système.

La classe **User** représente les utilisateurs de la plateforme et contient les informations liées à l'authentification et à la gestion du profil utilisateur. La classe **Compte** permet de gérer les différents comptes financiers associés à un utilisateur. Elle centralise également les opérations liées aux revenus, dépenses et transferts. Les classes **Income** et **Outcome** représentent respectivement les revenus et les dépenses enregistrés par l'utilisateur. Chaque opération est liée à un compte ainsi qu'à une sous-catégorie spécifique afin de faciliter l'organisation des données financières. Les classes **Category** et **SousCateg** permettent de structurer les différentes catégories budgétaires utilisées dans l'application. La classe **Budget** assure le suivi des limites budgétaires définies par l'utilisateur pour une catégorie donnée, selon différentes périodes (mensuelle ou annuelle). Elle est directement liée à l'utilisateur ainsi qu'à la catégorie qu'elle surveille, permettant de générer des alertes en cas de dépassement. Les classes **Note** et **NikkiEntry** offrent des fonctionnalités complémentaires permettant la gestion des notes personnelles, checklists et entrées de journal. La classe **NikkiEntryImage** permet quant à elle d'associer plusieurs images à une entrée de journal.

Cette architecture modulaire garantit une meilleure organisation du système, une séparation claire des responsabilités ainsi qu'une bonne maintenabilité du projet.

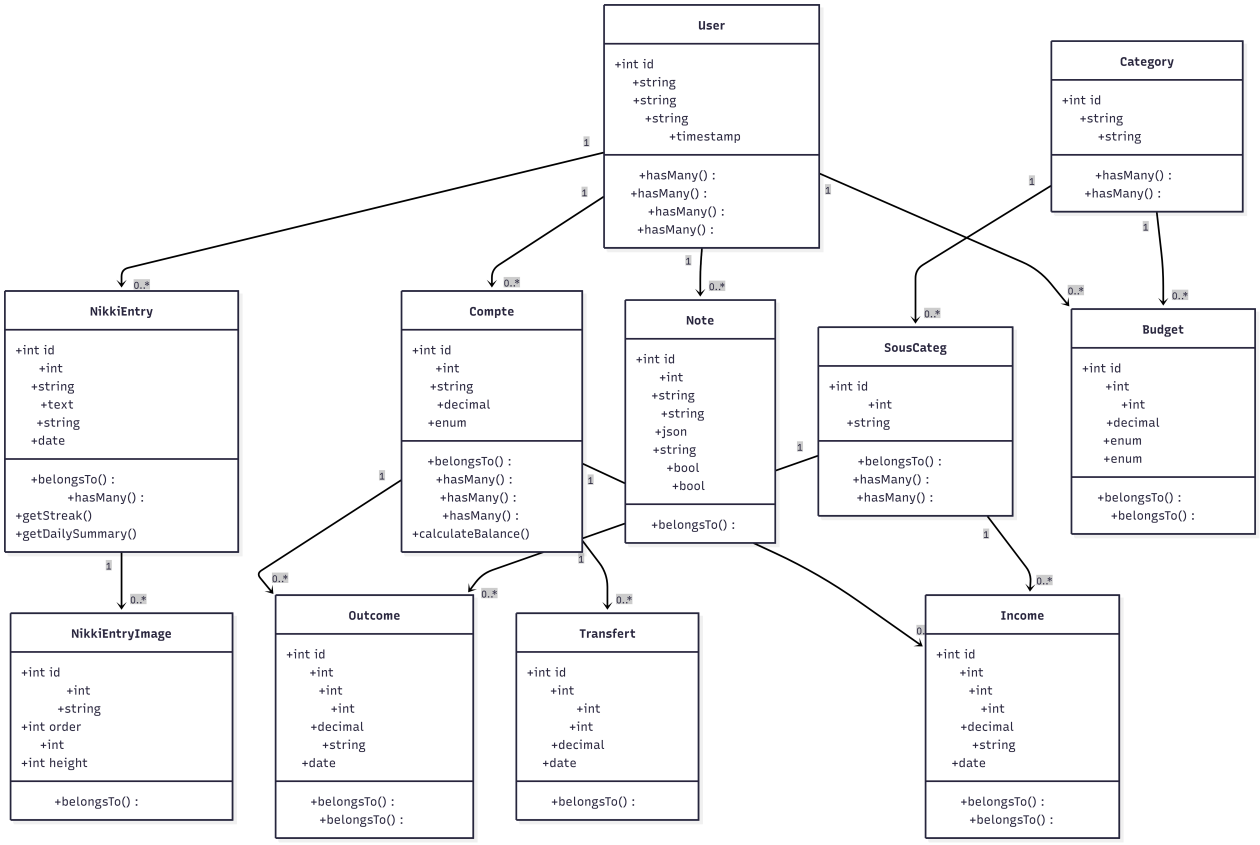


FIGURE 2.8 – Diagramme de classes structurel du système

Ce modèle conceptuel de données explicite l'organisation des tables logiques. On note l'héritage ou la relation des flux financiers (Income/Outcome) reliés à leurs catégories respectives, ainsi que le découpage modulaire pour le journal.

2.6 Conclusion

Ce chapitre a permis de présenter les différentes étapes d'analyse et de conception réalisées avant la phase de développement de l'application. Grâce à l'utilisation des diagrammes UML ainsi qu'aux maquettes réalisées sur Figma, les besoins fonctionnels et les interactions entre les différents composants du système ont pu être clairement identifiés et structurés.

Cette phase de modélisation a constitué une étape essentielle dans la conception de l'application en garantissant une architecture cohérente, maintenable et adaptée aux objectifs du projet.

Le chapitre suivant sera consacré à la réalisation technique de l'application, en présentant l'architecture adoptée, les technologies utilisées ainsi que les différentes fonctionnalités implémentées.

Chapitre 3

Réalisation Technique

3.1 Introduction

Après les phases d'analyse et de conception, la phase de réalisation a permis de concrétiser l'ensemble des fonctionnalités définies pour l'application. Cette étape consiste à mettre en œuvre l'architecture technique, développer les différentes fonctionnalités du système et assurer la communication entre le frontend et le backend.

Ce chapitre présente les choix techniques adoptés, les technologies utilisées ainsi que l'organisation générale de l'application.

3.2 Architecture générale du projet

L'application repose sur une architecture full-stack moderne séparant le frontend et le backend afin de garantir une meilleure organisation du code, une maintenance simplifiée ainsi qu'une meilleure évolutivité du système.

L'architecture du projet est composée des éléments suivants :

- Frontend développé avec **React** ;
- Backend développé avec **Laravel** ;
- Communication entre les deux parties via des **API REST** ;
- Base de données **MySQL** utilisée durant le développement ;
- Authentification sécurisée avec **Laravel Sanctum** ;
- Service externe **Groq API** sollicité pour le module d'analyse financière par intelligence artificielle.

Cette séparation permet de gérer indépendamment l'interface utilisateur et la logique métier tout en assurant des échanges rapides et sécurisés entre les différentes composantes du système. Le backend agit comme point central de l'application : il gère l'authentification via Sanctum, interroge la base de données MySQL pour les opérations courantes, et effectue, de manière indépendante, un appel vers l'API externe Groq lorsque l'utilisateur sollicite une analyse financière par IA. Ces deux communications (MySQL et Groq) sont distinctes et n'interfèrent pas l'une avec l'autre.

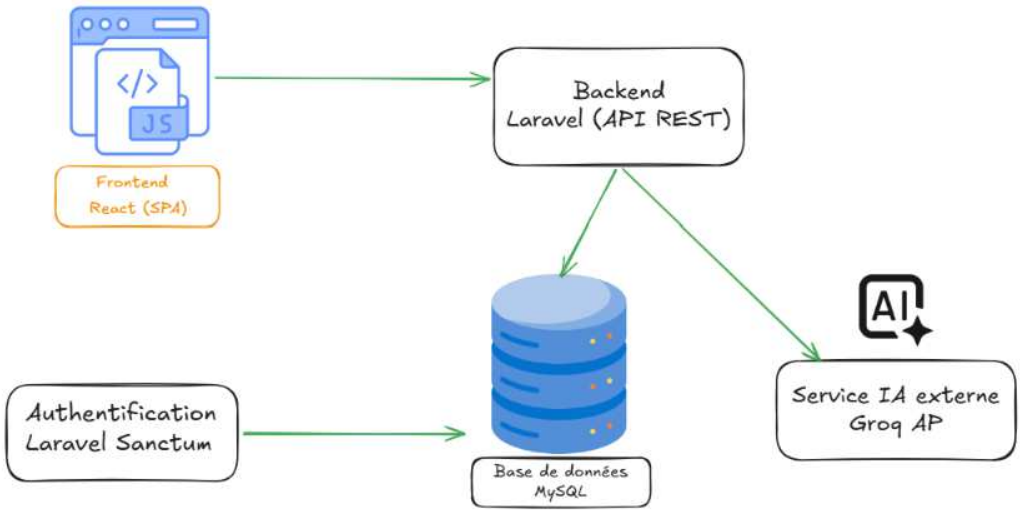


FIGURE 3.9 – Architecture générale de l'application (Client-Serveur)

Ce schéma illustre l'architecture découplée de notre projet. Le Frontend (React) émet des requêtes asynchrones via HTTPS vers les points de terminaison (endpoints) du Backend (Laravel API), sécurisés par un middleware d'authentification Sanctum. Le Backend communique de manière indépendante avec la base de données MySQL pour les opérations courantes, ainsi qu'avec le service externe Groq, hébergeant le modèle `llama-3.3-70b-versatile`*, pour générer les analyses financières du module d'intelligence artificielle.*

3.3 Outils, langages et technologies utilisés

Plusieurs technologies et outils ont été utilisés durant le développement du projet afin de garantir la performance, la sécurité et la qualité globale de l'application.

3.3.1 React.js

React.js est une bibliothèque JavaScript open-source développée par Meta, utilisée pour le développement d'interfaces utilisateur dynamiques, réactives et interactives basées sur des composants réutilisables. Dans ce projet, React a été utilisé pour développer l'intégralité du frontend de l'application, notamment le tableau de bord en temps réel, les formulaires de saisie de transactions, le rendu des statistiques financières ainsi que les différents composants d'interface.

3.3.2 Laravel



Laravel est un framework PHP robuste et moderne basé sur l'architecture MVC (Modèle-Vue-Contrôleur). Il intègre de puissants outils natifs comme l'ORM Eloquent pour simplifier les interactions avec la base de données. Il a été choisi pour concevoir le backend de l'application, piloter la logique métier globale, structurer les contrôleurs des API REST, sécuriser l'accès aux routes et orchestrer la gestion sécurisée des profils utilisateurs.

3.3.3 Tailwind CSS



Tailwind CSS est un framework CSS orienté "utility-first" qui permet d'appliquer des styles directement au sein du balisage HTML ou JSX à l'aide de classes prédéfinies. Il a été exploité pour l'intégration visuelle de l'application et la création d'un design moderne, assurant une parfaite réactivité (*responsiveness*) des interfaces sur les écrans d'ordinateurs, de tablettes et de smartphones.

3.3.4 Axios



Axios est une bibliothèque JavaScript basée sur les Promesses, configurée pour intercepter et gérer les requêtes HTTP depuis le navigateur. Dans ce projet, Axios fait office de passerelle de communication, permettant au frontend React de consommer de manière asynchrone les données JSON exposées par les routes de l'API Laravel.

3.3.5 Laravel Sanctum



Laravel Sanctum fournit un système d'authentification léger pour les SPA (Single Page Applications), les applications mobiles et les API simples basées sur des jetons d'accès (*API tokens*). Il a été implémenté pour sécuriser les points d'accès de l'API, en délivrant un jeton d'accès unique à l'utilisateur lors de sa connexion pour valider chacune de ses requêtes ultérieures.

3.3.6 Lucide React



Lucide React est une bibliothèque d'icônes open source destinée aux applications React. Elle a été utilisée pour enrichir l'interface utilisateur en intégrant des icônes modernes, cohérentes et facilement personnalisables, contribuant ainsi à améliorer la lisibilité et l'expérience utilisateur.

3.3.7 MySQL



MySQL est un moteur de base de données relationnelle autonome, sans serveur et ne nécessitant aucune configuration complexe, stockant l'intégralité des données dans un simple fichier physique. Durant la phase de développement de MoneyIO, MySQL a été choisi pour assurer le stockage persistant et rapide des informations relatives aux utilisateurs, transactions, budgets et notes.

3.3.8 Postman



Postman est une plateforme permettant de tester et de manipuler des API REST. Dans ce projet, il a été utilisé pour vérifier le bon fonctionnement des contrôleurs Laravel, tester les différentes requêtes HTTP et valider les réponses de l'API avant leur intégration avec le frontend React.

3.3.9 Laravel Telescope



Laravel Telescope est un outil de débogage et de supervision natif à l'écosystème Laravel, offrant une interface visuelle permettant d'inspecter les requêtes, les jobs, les exceptions et les logs de l'application. Il a été utilisé durant le développement afin de surveiller le comportement de l'API en temps réel, faciliter le diagnostic des erreurs et optimiser les performances des requêtes SQL.

3.3.10 TipTap



TipTap est un framework d'éditeur de texte riche basé sur ProseMirror, offrant une architecture extensible par système de plugins. Il a été intégré au module Journal Nikki afin de permettre la rédaction d'entrées enrichies (gras, italique, couleurs, alignement, images), ainsi que le développement d'extensions personnalisées telles que le redimensionnement d'images et le système de mentions de transactions.

3.3.11 Git



Git est un système de gestion de versions décentralisé permettant de suivre l'historique des modifications du code source et de faciliter le travail collaboratif. Il a été utilisé tout au long du projet afin de versionner le code du frontend et du backend, gérer les différentes branches de développement et assurer une sauvegarde fiable et traçable de l'évolution de l'application.

3.3.12 Figma



Figma est un outil collaboratif en ligne de conception graphique et de prototypage d'interfaces pour le design d'applications (UI/UX). Cet outil a permis de concevoir l'ergonomie, la charte graphique et le comportement interactif des écrans de l'application en amont du codage, servant de fil conducteur pour le développement de l'interface.

3.3.13 Visual Studio Code



Visual Studio Code est un éditeur de code source extensible et léger développé par Microsoft, intégrant un terminal et un gestionnaire Git. Il a été utilisé comme environnement de développement intégré (IDE) principal pour l'écriture, le débogage et la refactorisation du code source frontend et backend.

3.4 Gestion de l'authentification et de la sécurité

La sécurité de l'application constitue un aspect important du projet. Pour cela, Laravel Sanctum a été utilisé afin de sécuriser les routes API et gérer l'authentification des utilisateurs.

Lors de la connexion, un token d'authentification unique (jeton d'accès) est généré par le serveur. Ce jeton est conservé côté client et transmis dans les en-têtes de chaque requête HTTP ultérieure pour accéder aux fonctionnalités protégées.

De plus, des mécanismes de validation de formulaires stricts (*Form Requests* de Laravel) ont été mis en place côté backend afin de vérifier l'intégrité et la conformité de chaque donnée saisie avant son insertion en base de données, bloquant ainsi les injections ou requêtes malveillantes.

3.5 Communication entre le frontend et le backend

La communication entre React et Laravel repose sur l'architecture d'API REST. Le frontend envoie des requêtes HTTP (via les verbes `GET`, `POST`, `PUT`, `DELETE`) vers le backend afin de manipuler les ressources numériques du système.

Les principales opérations réalisées via l'API sont :

— L'authentification sécurisée des sessions utilisateurs ;

— La création, modification et suppression des comptes financiers ;

— L'enregistrement en temps réel des revenus, dépenses et virements ;

— Le suivi et l'évaluation dynamique des limites de budgets ;

— Le calcul et la récupération des statistiques financières consolidées ;

— La synchronisation des mémos, des checklists et des récits du journal personnel.

Cette architecture découplée assure une séparation stricte des responsabilités : le backend gère exclusivement la donnée et la sécurité, tandis que le frontend prend en charge l'expérience et l'affichage utilisateur.

3.6 Conclusion

Ce chapitre a présenté les principaux choix techniques adoptés pour la réalisation de l'application ainsi que les technologies et outils utilisés tout au long du développement.

L'architecture retenue, basée sur React pour le frontend et Laravel pour le backend, a permis de concevoir une application moderne, modulaire et évolutive. La communication via des API REST sécurisées par Laravel Sanctum assure des échanges fiables entre les différentes composantes du système, tandis que les outils de développement utilisés ont contribué à garantir la qualité et la maintenabilité du projet.

Le chapitre suivant sera consacré à la présentation des principales fonctionnalités de l'application à travers une démonstration illustrée des différentes interfaces réalisées.

Chapitre 4

Réalisation et Démonstration de l'Application

4.1 Introduction

Ce quatrième chapitre est dédié à la présentation concrète de l'application de gestion financière **MoneyIO** développée durant ce stage. Après avoir défini les besoins fonctionnels et l'architecture technique du système dans les chapitres précédents, nous exposerons ici l'interface utilisateur (UI) finale ainsi que l'expérience utilisateur (UX) qui en découle. À travers une série de captures d'écran issues de l'environnement d'exécution local, nous détaillerons le fonctionnement de chaque module, allant du tableau de bord principal aux outils avancés d'intelligence artificielle et de journalisation financière contextuelle.



FIGURE 4.10 – Logo de l'application MoneyIO

4.2 Landing Page

Avant toute connexion, l'utilisateur non authentifié accède à une page de présentation (*Landing Page*) conçue pour présenter MoneyIO, mettre en avant ses principales fonctionnalités et inciter les visiteurs à créer un compte ou à se connecter. Cette page constitue la vitrine de l'application et permet de découvrir rapidement son fonctionnement.

4.2.1 En-tête et présentation de l'application

La partie supérieure de la page présente l'identité visuelle de MoneyIO ainsi que son principal slogan : *« Votre argent, enfin clair. »*. Elle contient également une courte description de l'application, les boutons **Créer un compte gratuit** et **Se connecter**, ainsi que le sélecteur de langue permettant de basculer entre le français et l'anglais.

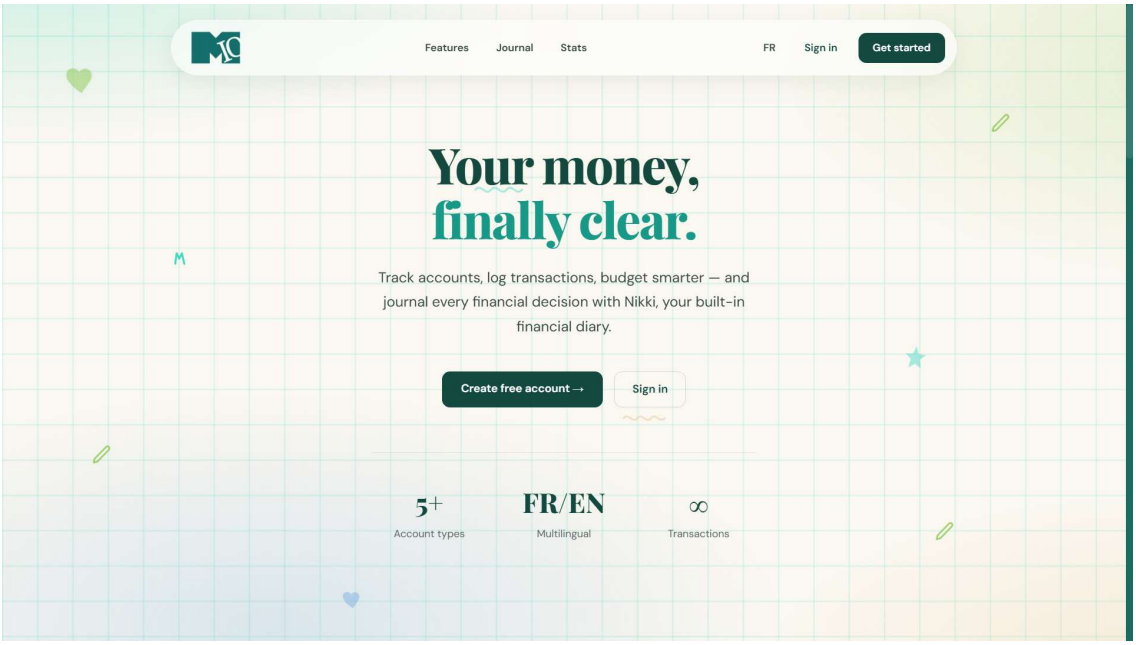


FIGURE 4.11 – En-tête de la Landing Page

4.2.2 Présentation des fonctionnalités

Cette section met en avant les principaux modules disponibles dans l'application sous forme de cartes illustrées. Chaque carte présente brièvement une fonctionnalité telle que le tableau de bord, le grand livre, les budgets, les notes, le journal Nikki ou encore les statistiques.

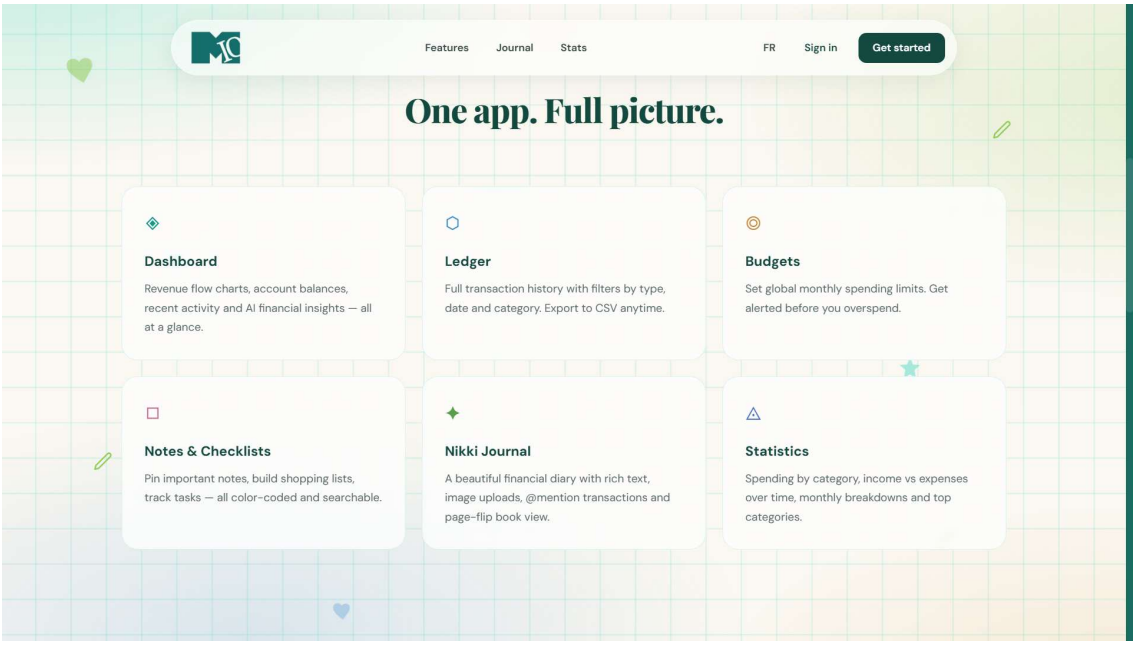


FIGURE 4.12 – Présentation des fonctionnalités principales

4.2.3 Présentation du journal Nikki

Afin de mettre en valeur l'une des fonctionnalités principales de MoneyIO, une section est entièrement consacrée au journal financier Nikki. Celle-ci présente ses principales caractéristiques, notamment l'éditeur de texte enrichi, la gestion des images, les mentions de transactions, la vue sous forme de livre ainsi que les différents thèmes disponibles.

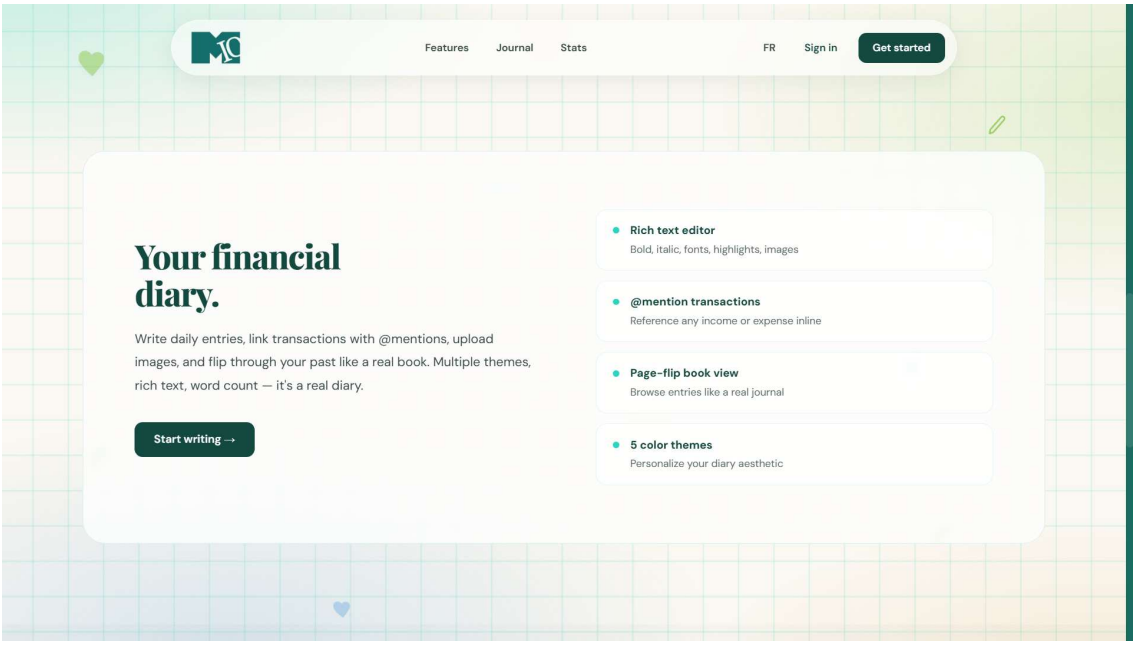


FIGURE 4.13 – Présentation du journal Nikki

4.2.4 Formulaire de contact

La Landing Page intègre également un formulaire permettant aux visiteurs de contacter directement les administrateurs de l'application sans avoir besoin de créer un compte. Les utilisateurs peuvent saisir leur nom, leur adresse électronique ainsi que leur message.

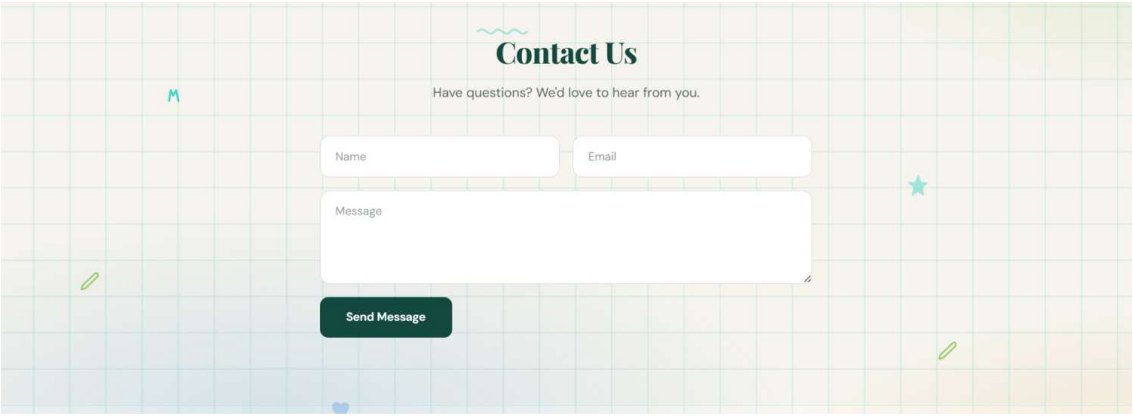


FIGURE 4.14 – Formulaire de contact

4.2.5 Appel à l'action final

La dernière partie de la Landing Page encourage une dernière fois les visiteurs à créer gratuitement un compte MoneyIO. Elle est suivie du pied de page contenant les liens de navigation ainsi que les informations générales de l'application.

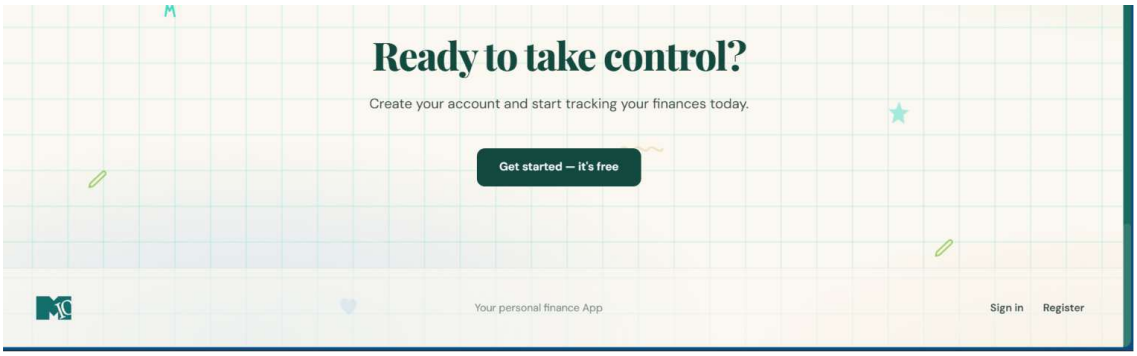


FIGURE 4.15 – Appel à l'action final et pied de page

4.3 Authentification

L'accès à l'application est entièrement sécurisé par un système d'authentification robuste protégeant les données personnelles et financières de l'utilisateur.



FIGURE 4.16 – Interface d'authentification (Connexion / Inscription)

Cette interface unifiée propose deux onglets basculables :

— **Sign in (Connexion) :** L'utilisateur renseigne son e-mail et son mot de passe pour accéder à son espace personnel. Un bouton `Show` permet d'afficher ou de masquer temporairement le mot de passe saisi. En cas d'absence de compte, un lien `Create one` redirige directement vers le formulaire d'inscription.

— **Create account (Inscription) :** Permet à un nouvel utilisateur de créer un compte en renseignant un nom d'utilisateur, une adresse e-mail valide et un mot de passe sécurisé. C'est à cette étape qu'une instance de base de données personnelle est initialisée pour l'utilisateur, lui permettant de configurer ultérieurement ses préférences de devise et de thème visuel.

Une fois l'authentification réussie, un jeton de session sécurisé (via Laravel Sanctum) est généré, permettant de maintenir l'état connecté de l'utilisateur et de le rediriger automatiquement vers son tableau de bord.

4.4 Tableau de bord

Le tableau de bord constitue le cœur décisionnel de l'application MoneyIO. Il offre une vue d'ensemble holistique sur la santé financière globale de l'utilisateur.

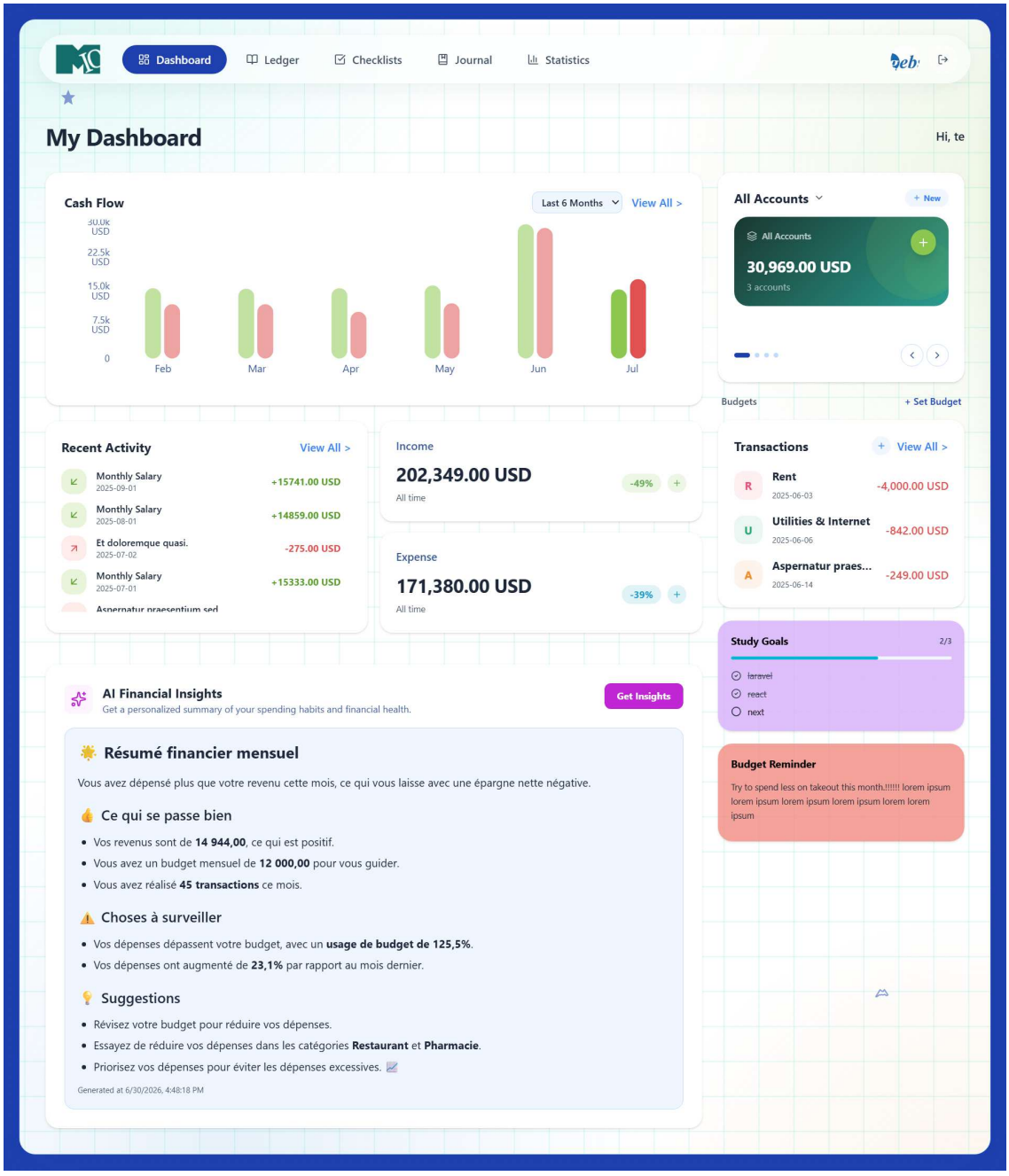


FIGURE 4.17 – Interface principale du Tableau de bord

L'écran intègre plusieurs widgets stratégiques disposés de manière ergonomique :

— **Graphique de Flux de Trésorerie (Cash Flow) :** Un histogramme dynamique présente la comparaison mensuelle entre les flux entrants et sortants sur les six derniers mois, facilitant l'analyse visuelle des tendances d'épargne.

— **Indicateurs Globaux :** Les encadrés centraux affichent de manière claire les totaux cumulés des revenus et des dépenses (*All time*), accompagnés d'indicateurs de croissance en

pourcentage.

— **Activités et Transactions Récentes :** Deux sections permettent de suivre en temps réel les dernières opérations enregistrées (salaires, abonnements, factures) avec une distinction de couleur pour une scannabilité optimale.

— **Barre Latérale de Navigation & Widgets Complémentaires :** À droite, l'interface affiche le carrousel des comptes bancaires de l'utilisateur, l'état d'avancement des objectifs d'études (*Study Goals*) sous forme de liste de contrôle gamifiée, et le rappel des limites budgétaires (*Budget Reminder*).

4.5 Gestion des comptes

La flexibilité financière de la plateforme repose sur une structure multi-comptes. Accessible depuis le widget présent sur le tableau de bord, la section **All Accounts** permet à l'utilisateur de centraliser et de segmenter ses capitaux (par exemple : compte courant, compte d'épargne ou portefeuille physique). L'interface offre un carrousel fluide affichant le solde cumulé de tous les comptes ainsi que le solde individuel de chaque sous-compte. L'ajout d'un nouveau compte s'effectue via le bouton « `+ New` », ouvrant une modale de saisie intuitive.

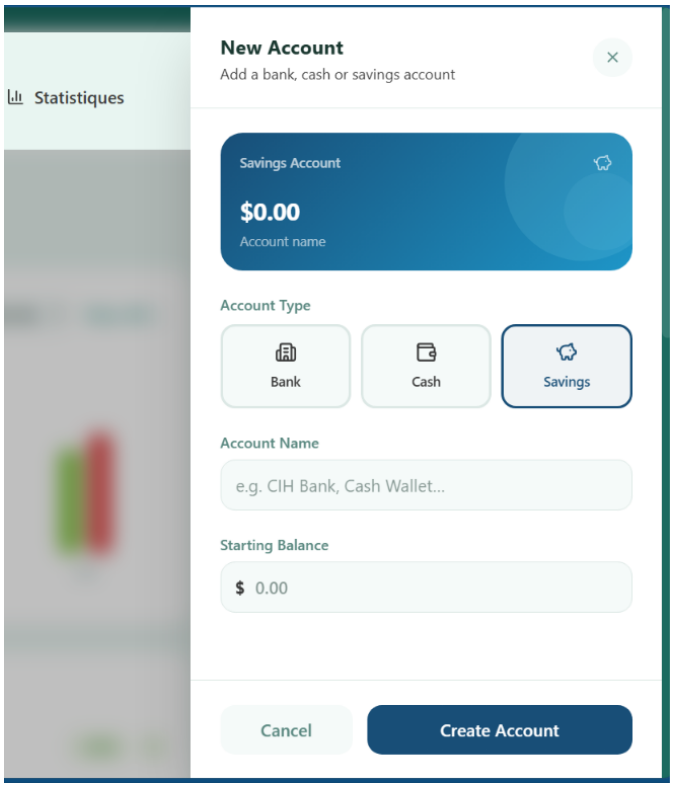


FIGURE 4.18 – Widget de gestion des comptes (All Accounts)

4.6 Gestion des transactions

Le système propose un registre modulaire permettant de catégoriser finement chaque flux monétaire. Une fenêtre modale unifiée permet à l'utilisateur d'enregistrer trois types d'opérations financières : un revenu, une dépense ou un virement entre comptes.

4.6.1 Revenus

Les transactions entrantes (crédits) sont représentées visuellement avec un code couleur vert et un signe positif (ex : `+202,349.00 EUR`). Elles permettent d'alimenter les différents comptes de l'utilisateur et d'ajuster dynamiquement sa capacité globale de dépenses.

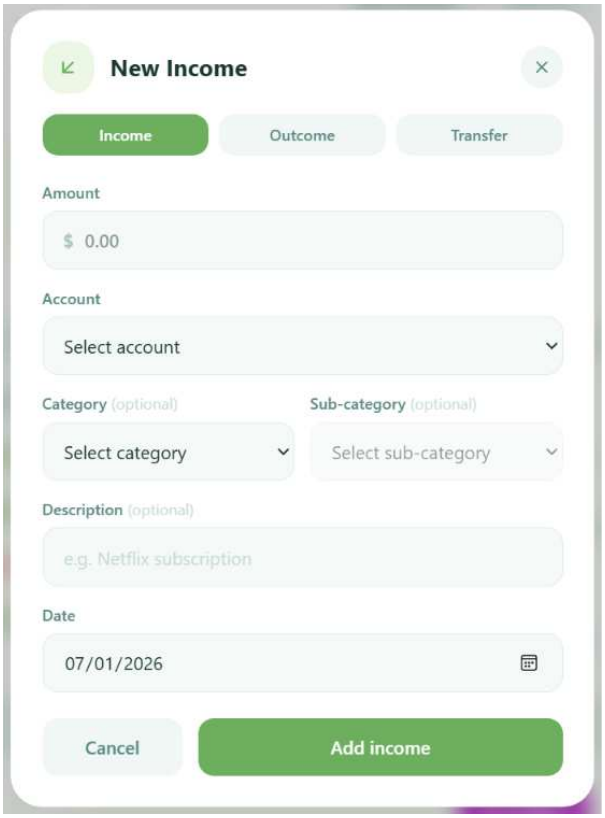


FIGURE 4.19 – Modal d'ajout d'un revenu

4.6.2 Dépenses

Les transactions sortantes (débits) apparaissent en rouge avec un signe négatif (ex : `-171,380.00 EUR`). Chaque dépense est obligatoirement associée à un compte source et à une étiquette de catégorie, essentielle pour alimenter l'outil statistique.

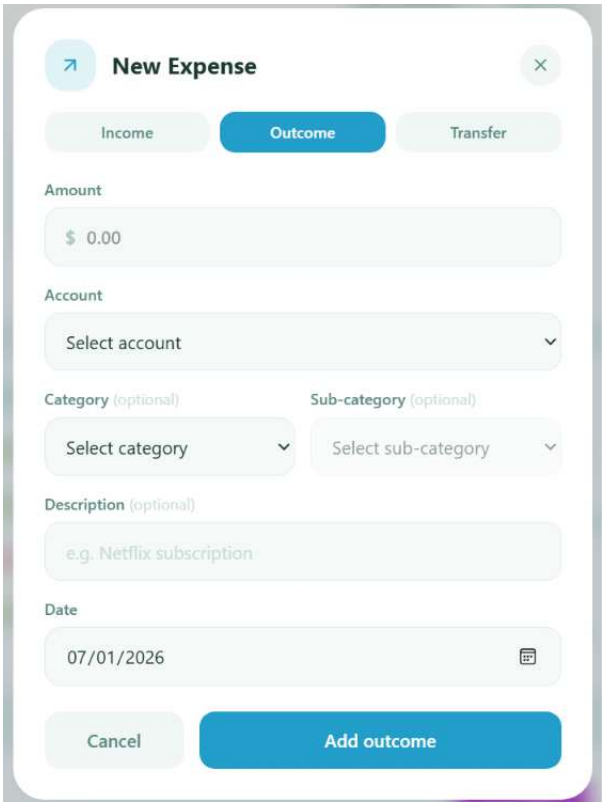


FIGURE 4.20 – Modal d'ajout d'une dépense

4.6.3 Virements

L'application supporte les transferts internes (virements de compte à compte). Ce type de transaction permet de déplacer des fonds de manière transparente sans altérer le montant net global des actifs de l'utilisateur. L'utilisateur sélectionne le compte source et le compte destinataire, renseigne le montant ainsi que la date du transfert ; les soldes des deux comptes concernés sont automatiquement mis à jour après validation.

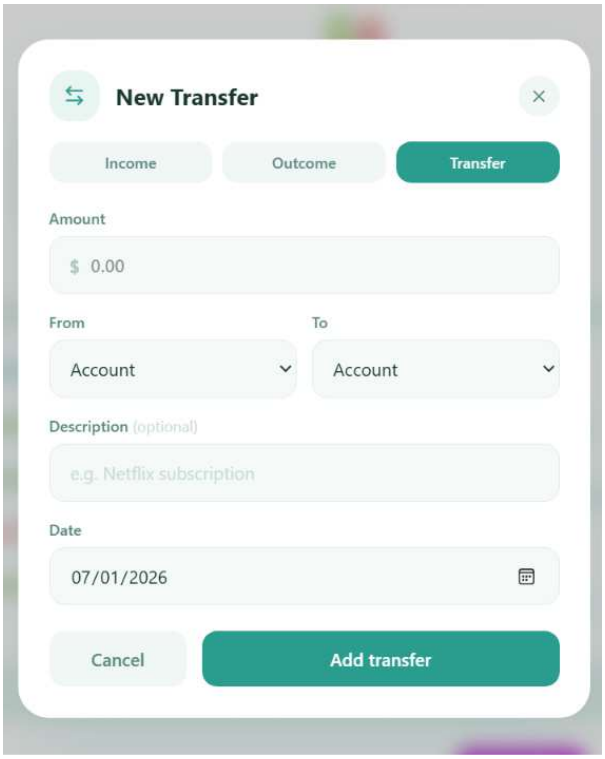


FIGURE 4.21 – Modal de virement entre comptes

4.7 Ledger (Grand livre)

Le **Grand livre** fait office de journal comptable exhaustif. En haut de l'interface, des cartes de synthèse affichent en temps réel les indicateurs clés : le total des **Revenus**, le total des **Dépenses**, et le solde **Net** disponible.

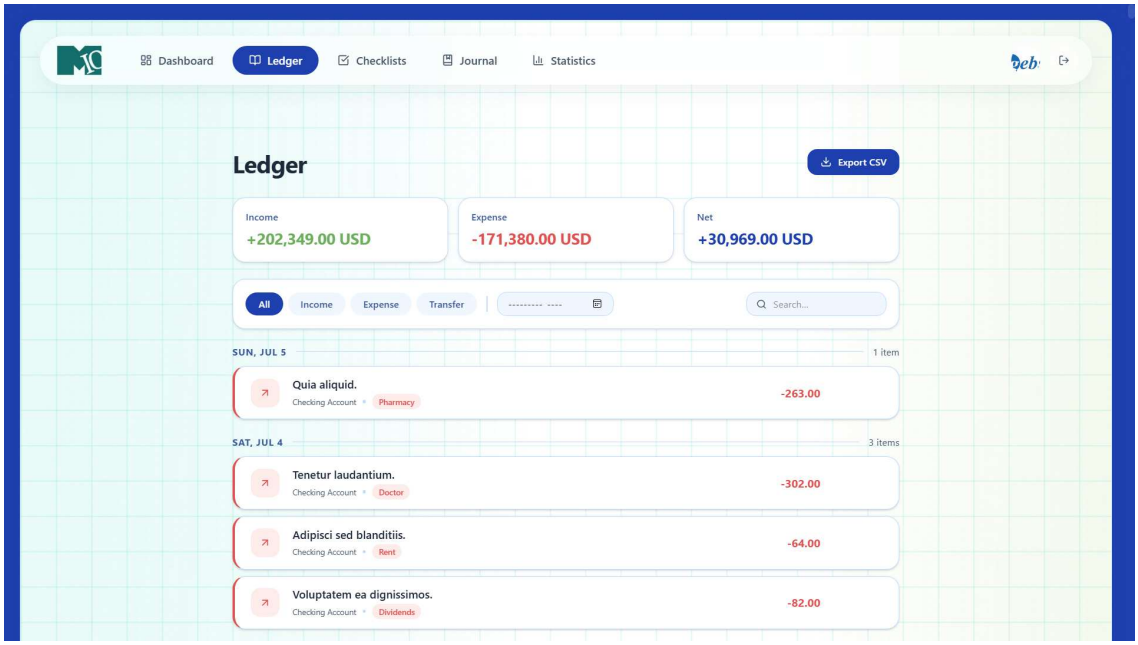


FIGURE 4.22 – Grand livre des transactions

L'utilisateur dispose de filtres segmentés (*Tout, Income, Expense, Transfer*) ainsi que d'un sélecteur de plages de dates et d'une barre de recherche contextuelle pour auditer instantanément ses transactions. De plus, un bouton de raccourci « `Exporter CSV` » permet l'extraction immédiate des données comptables pour un traitement sur tableur externe.

4.8 Budgets

L'application intègre un module de contrôle budgétaire avancé visant à prévenir le surendettement. Le suivi des budgets est matérialisé par des barres de progression visuelles visibles sur le tableau de bord et sur la page Statistiques.

Lorsqu'un budget mensuel global ou sectoriel subit un dépassement critique, la barre d'état vire automatiquement au rouge vif (comme illustré par l'alerte de budget à `125%` affichant un message explicite de dépassement : *Limit exceeded by -3,060.00 EUR*). Un système d'alerte dynamique (*Budget Reminder*) notifie l'utilisateur sur son tableau de bord afin de l'inciter à restreindre ses dépenses non essentielles.

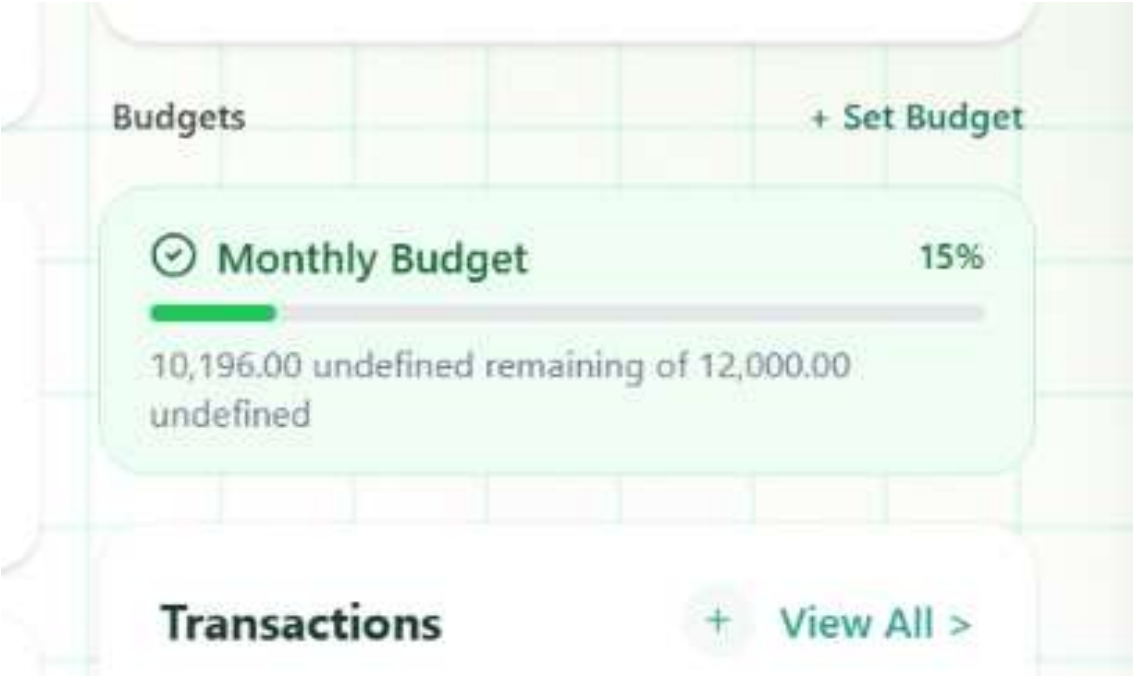


FIGURE 4.23 – Alerte de dépassement de budget

4.9 Statistiques

La page **Statistiques** centralise l'ensemble des indicateurs analytiques permettant à l'utilisateur d'évaluer sa santé financière sur une période donnée.

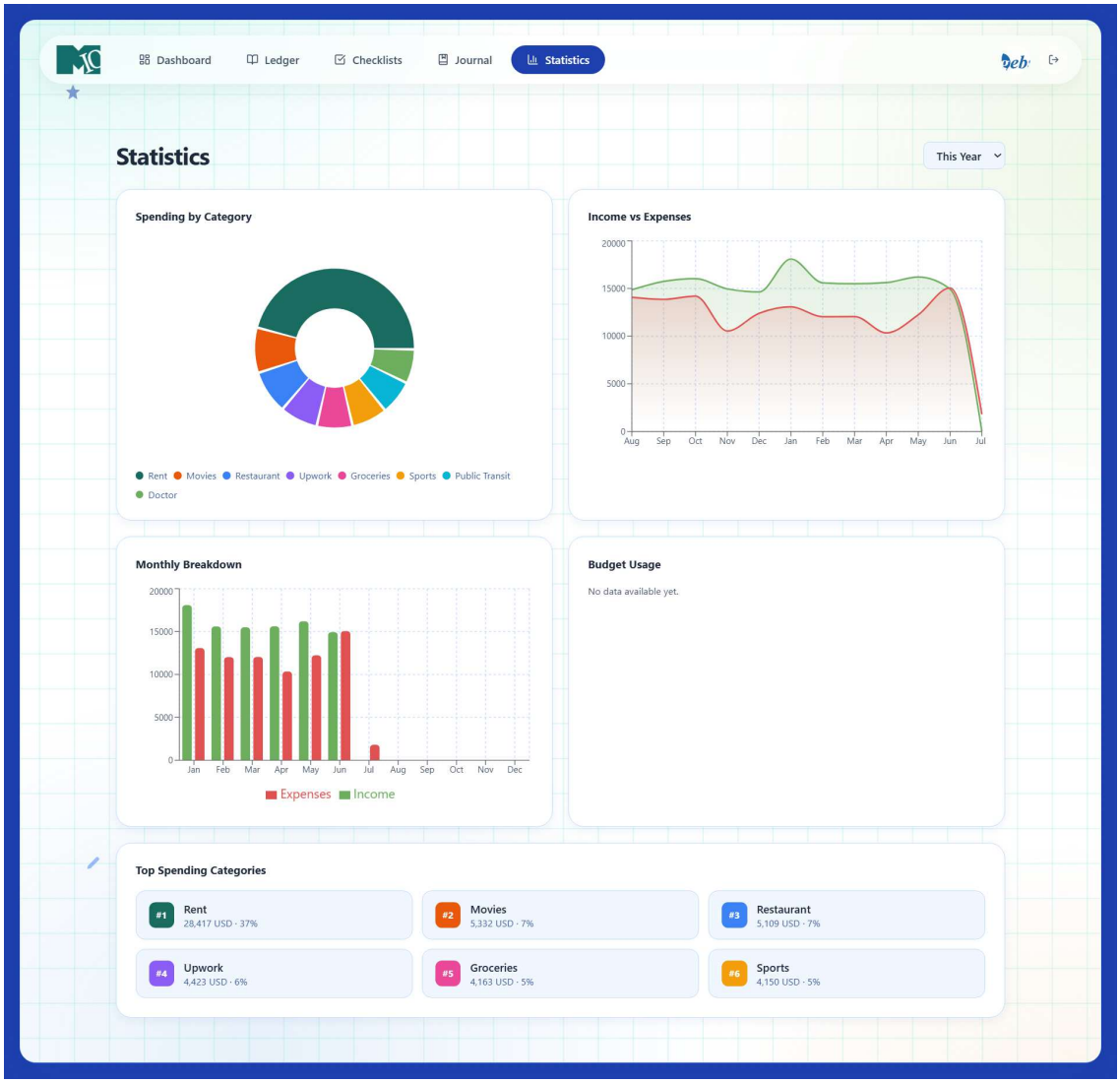


FIGURE 4.24 – Page Statistiques

Cette page regroupe plusieurs visualisations complémentaires :

— **Dépenses par catégorie :** Un graphique en anneau (*donut chart*) répartit les dépenses selon les catégories (Rent, Movies, Restaurant, Upwork, Groceries, Sports, Public Transit, Doctor).

— **Revenus vs Dépenses :** Un graphique en aires superposées compare l'évolution mensuelle des revenus et des dépenses sur les douze derniers mois.

— **Répartition mensuelle :** Un histogramme comparatif affiche, mois par mois, le total des dépenses et des revenus.

— **Utilisation du budget :** Des barres de progression indiquent le pourcentage d'utilisation du budget global et par catégorie, avec alerte visuelle en cas de dépassement.

— **Top catégories de dépenses :** Un classement des six catégories les plus coûteuses, affi-

chant le montant total et le pourcentage correspondant.

4.10 IA financière

L'application se démarque par l'intégration d'un module d'analyse prédictive et décisionnelle baptisé **AI Financial Insights**, visible sur le tableau de bord. En exploitant les données de consommation historiques de l'utilisateur via un appel API asynchrone vers le modèle de langage `llama-3.3-70b-versatile` hébergé par Groq, le système génère instantanément un diagnostic personnalisé sous forme de rapport structuré en Markdown.

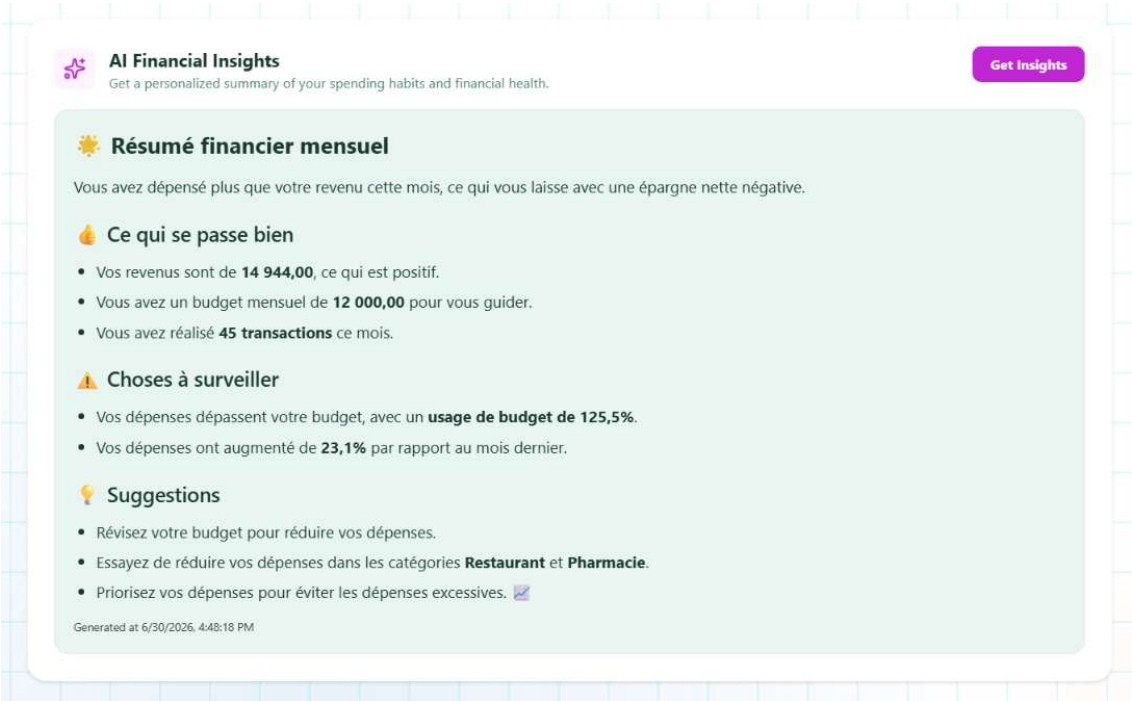


FIGURE 4.25 – Module AI Financial Insights

Ce module se décompose en quatre axes analytiques :

— **Résumé financier mensuel :** Interprétation sémantique du solde mensuel (ex. : identification d'une épargne nette négative).

— **Points positifs (Ce qui se passe bien) :** Mise en valeur des indicateurs de stabilité, comme la réalisation d'un grand nombre de transactions ou le respect théorique du plan budgétaire.

— **Axe de vigilance (Choses à surveiller) :** Détection automatique des dérives budgétaires exprimées en pourcentages et calcul de la hausse des charges par rapport au mois précédent.

— **Suggestions concrètes :** Recommandations algorithmiques ciblées (révision du budget global, restriction sur des catégories spécifiques comme le *Restaurant* ou la *Pharmacie*).

4.11 Notes et Checklists

Pour accompagner la gestion financière d'une organisation quotidienne, l'application propose un espace de prise de notes et de listes de tâches (Checklists) modulaire.

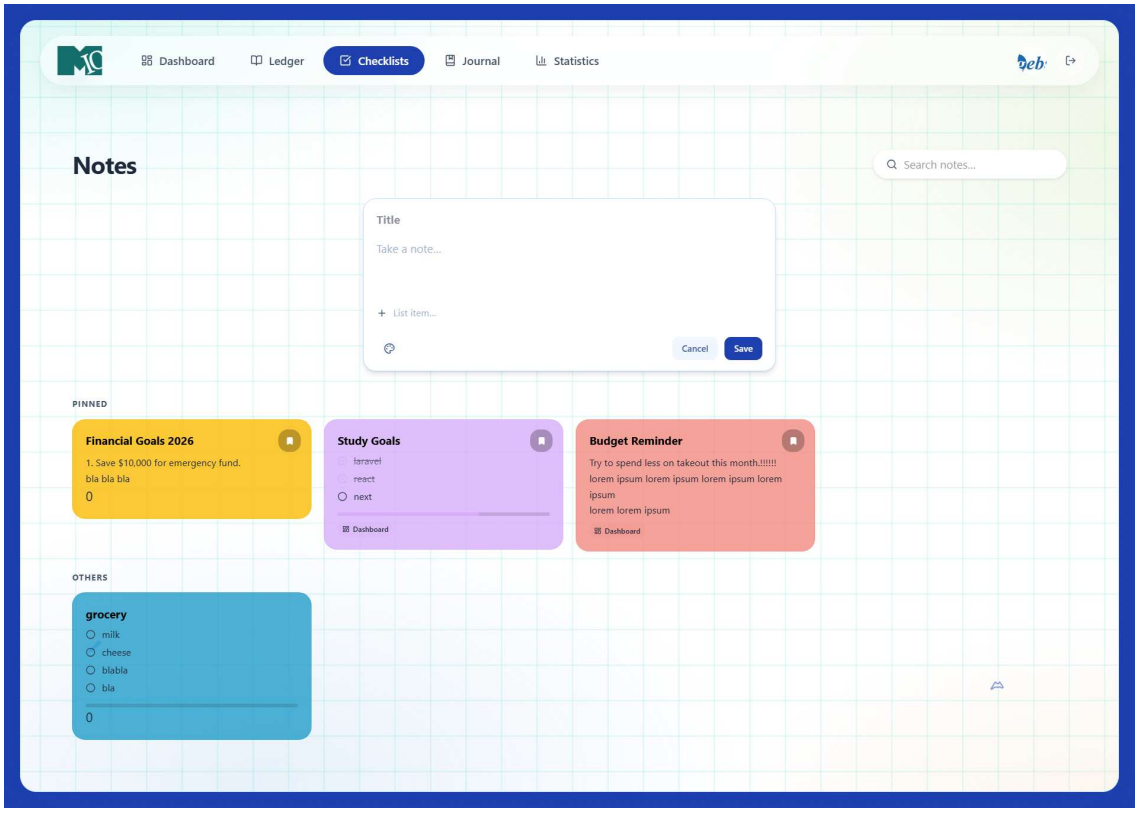


FIGURE 4.26 – Module de gestion des Notes et Checklists

Cet écran permet à l'utilisateur de concevoir des mémos à l'aide d'un éditeur de cartes colorées. Les notes importantes peuvent être épinglées en tête de page (*Pinned*) pour rester visibles. Chaque carte prend en charge l'intégration de listes d'éléments interactifs cochables avec des barres de complétion horizontales intégrées, idéales pour matérialiser le suivi d'objectifs académiques ou de rappels budgétaires complexes.

4.12 Journal Nikki

Le module **Nikki** est une innovation conceptuelle fusionnant le principe du journal intime traditionnel et le suivi comptable contextuel.

4.12.1 Accueil

À l'ouverture du module, l'utilisateur est accueilli par une interface épurée simulant la couverture fermée d'un livre physique minimaliste portant la mention *« nikki - your financial diary »*. Cette approche renforce l'aspect immersif et confidentiel du module.

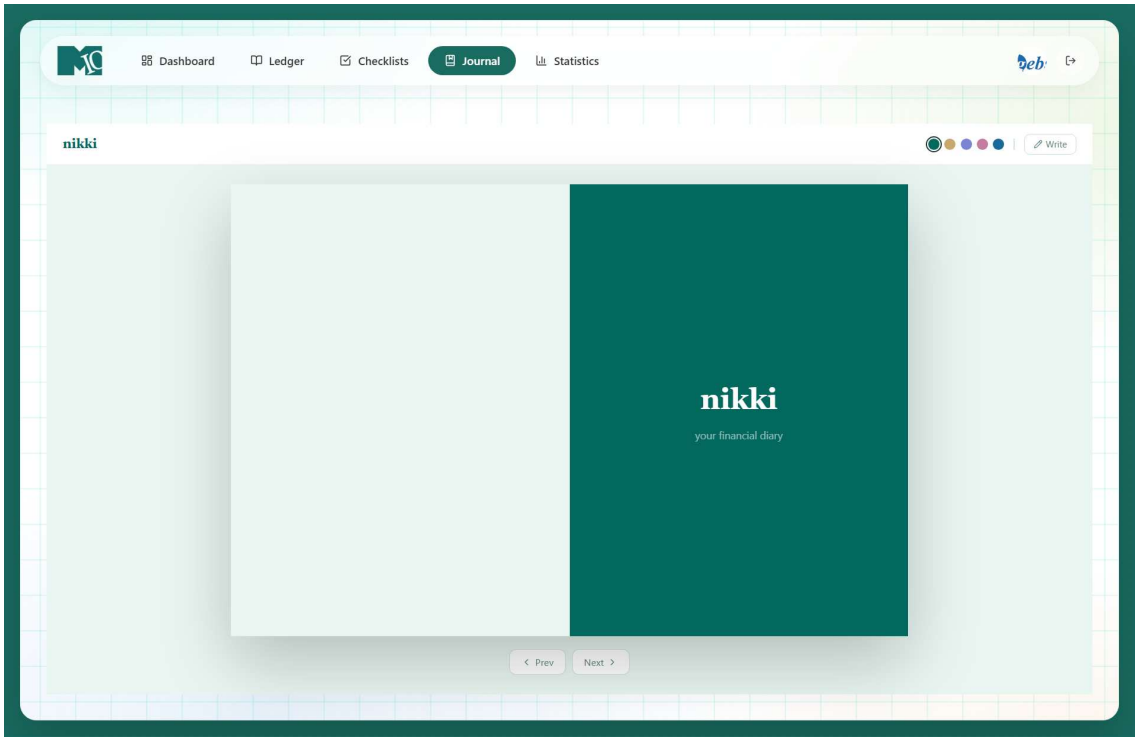


FIGURE 4.27 – Écran d'accueil du journal financier Nikki

4.12.2 Vue Livre et Effet Page-Flip

Une fois le journal ouvert, l'application bascule sur une interface simulant un livre ouvert à double-page avec un effet de transition réaliste, comme illustré par la figure 4.28.

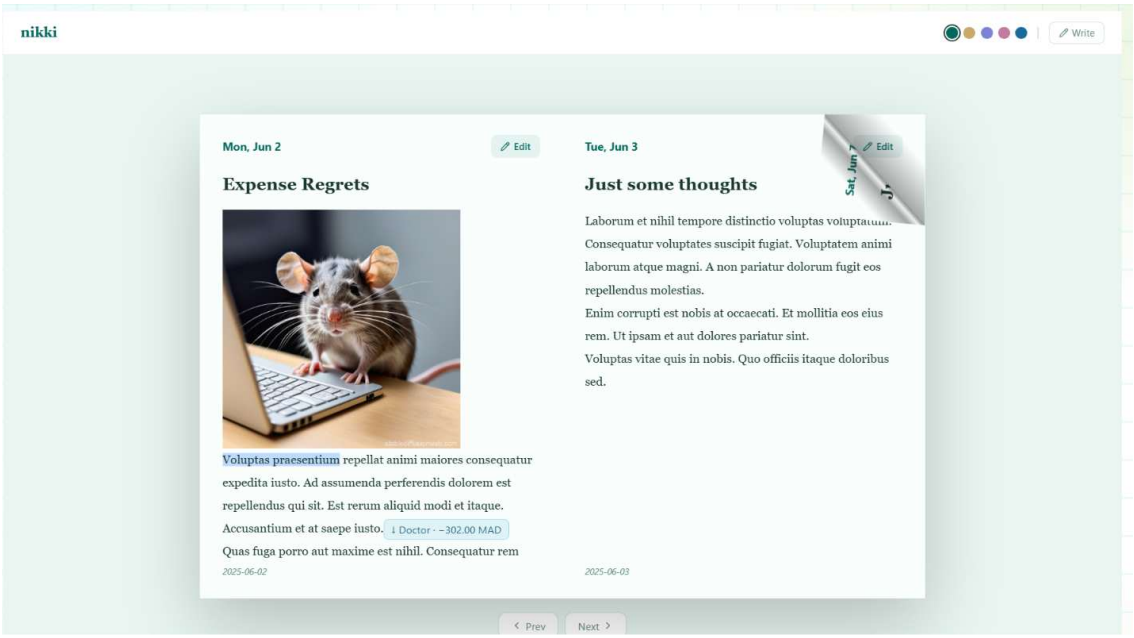


FIGURE 4.28 – Consultation double-page et effet d'animation page-flip

Cette vue s'organise autour de deux éléments clés :

— **Structure en double-page :** Les notes s'affichent côte à côte de manière chronologique (page de gauche et page de droite) pour offrir une lecture naturelle de type carnet.

— **Animation Page-Flip :** Le coin supérieur droit de la page intègre un effet visuel de feuille cornée interactive qui s'anime au survol ou au clic, permettant de feuilleter le journal comme un vrai livre physique.

4.12.3 Création d'une entrée

L'interface de rédaction de billets écrits est hautement perfectionnée, comme l'illustre la capture ci-dessous.

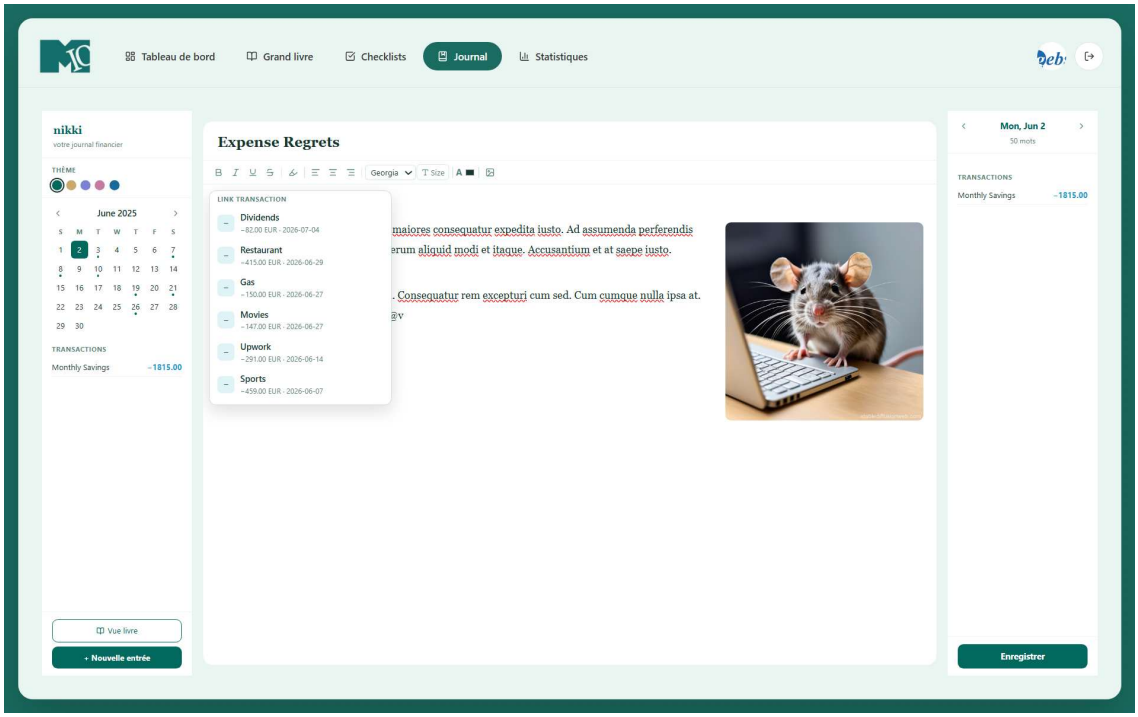


FIGURE 4.29 – Interface de création et d'édition d'une entrée Nikki

Cette interface regroupe plusieurs fonctionnalités clés :

— **Éditeur de texte enrichi (TipTap) :** Permet d'appliquer des styles typographiques avancés (gras, italique, souligné, barré), de choisir la police et d'ajuster la taille du texte.

— **Alignements et mise en forme :** L'utilisateur peut aligner le texte à gauche, au centre ou à droite, et structurer librement ses paragraphes.

— **Palettes colorimétriques d'écriture :** Un sélecteur de couleurs permet de personnaliser la teinte du texte rédigé.

— **Système de Mentions (@) :** En saisissant le caractère `@`, une fenêtre contextuelle (*Link Transaction*) s'ouvre, listant les transactions financières récentes de l'utilisateur. Ce dernier peut ainsi corréler instantanément un écrit à une dépense ou un gain réel, créant un lien hypertexte comptable au sein même de son récit.

— **Intégration d'images :** L'éditeur autorise l'ajout d'images par glisser-déposer (*Drag and Drop*) directement au sein de la page.

— **Calendrier latéral :** La barre latérale gauche affiche un calendrier mensuel interactif permettant de naviguer rapidement entre les différentes entrées par date.

— **Résumé quotidien :** Toujours dans la barre latérale, un encart compile automatiquement les indicateurs journaliers de la date sélectionnée (nombre de mots rédigés, montant cumulé des transactions liées).

— **Sélecteur de thème :** Un choix de cinq palettes de couleurs pastel permet de personnaliser

instantanément l'ambiance visuelle du journal.

4.13 Profil utilisateur

L'administration des informations nominatives s'opère sur la page dédiée au profil.

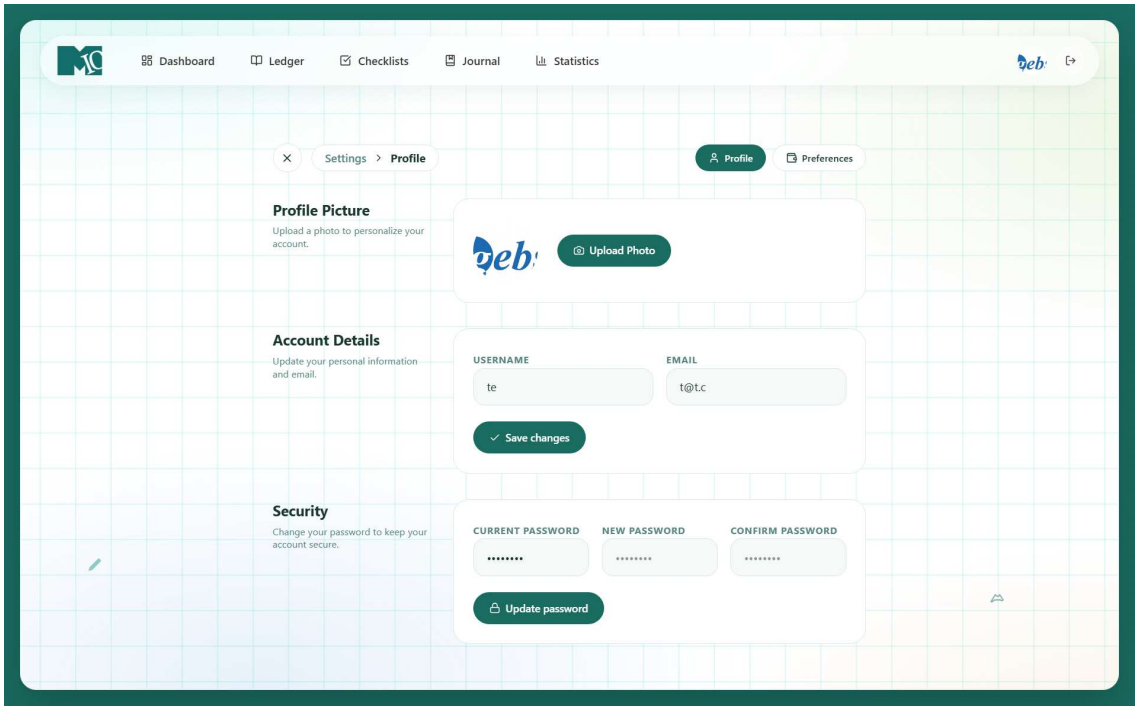


FIGURE 4.30 – Page de gestion du Profil Utilisateur

Cet écran permet d'effectuer trois actions fondamentales :

— **Photo de profil :** Téléchargement, mise à jour et recadrage d'un avatar personnalisé.

— **Détails du compte :** Modification à la volée du nom d'utilisateur et de l'adresse e-mail de correspondance.

— **Sécurité :** Formulaire de changement sécurisé du mot de passe avec vérification de la clé d'authentification actuelle.

4.14 Paramètres et Préférences

L'ajustement global du comportement de l'application est centralisé au sein de l'onglet **Préférences**.

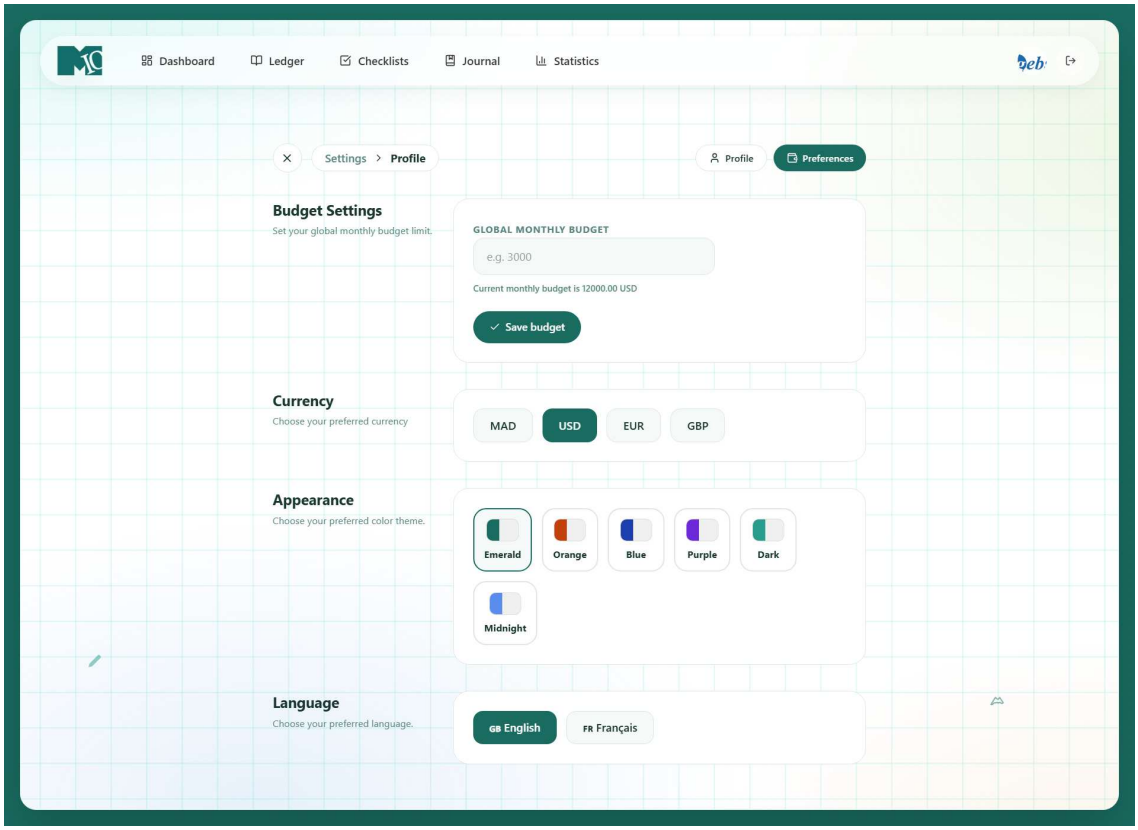


FIGURE 4.31 – Panneau des Préférences et Paramètres globaux

Cette section est segmentée en quatre axes de configuration :

— **Paramètres de budget :** Saisie du plafond monétaire pour le budget mensuel global (ex. : `12 000,00 EUR`).

— **Currency (Devise) :** Sélecteur d'unités monétaires prenant nativement en charge le Dirham Marocain (`MAD`), le Dollar Américain (`USD`), l'Euro (`EUR`), et la Livre Sterling (`GBP`).

— **Apparence (Thèmes globaux) :** Gestion du mode d'affichage via des variations de palettes graphiques (*Emerald, Orange, Blue, Purple, Dark, Midnight*) modifiant l'ensemble des ombres et couleurs de l'application.

— **Langue :** Support de l'internationalisation (*i18n*) permettant de basculer instantanément l'intégralité de l'interface entre l'Anglais (`GB English`) et le Français (`FR Français`).

4.15 Administration

En plus des fonctionnalités accessibles à tout utilisateur standard, un compte disposant du rôle **admin** a accès à un panneau d'administration dédié, matérialisé par un onglet **Admin** supplémentaire dans la barre de navigation. Cette interface centralise la supervision de la plateforme et

la gestion des comptes utilisateurs, conformément aux cas d'utilisation identifiés lors de la phase de conception (voir Figure 2.5).

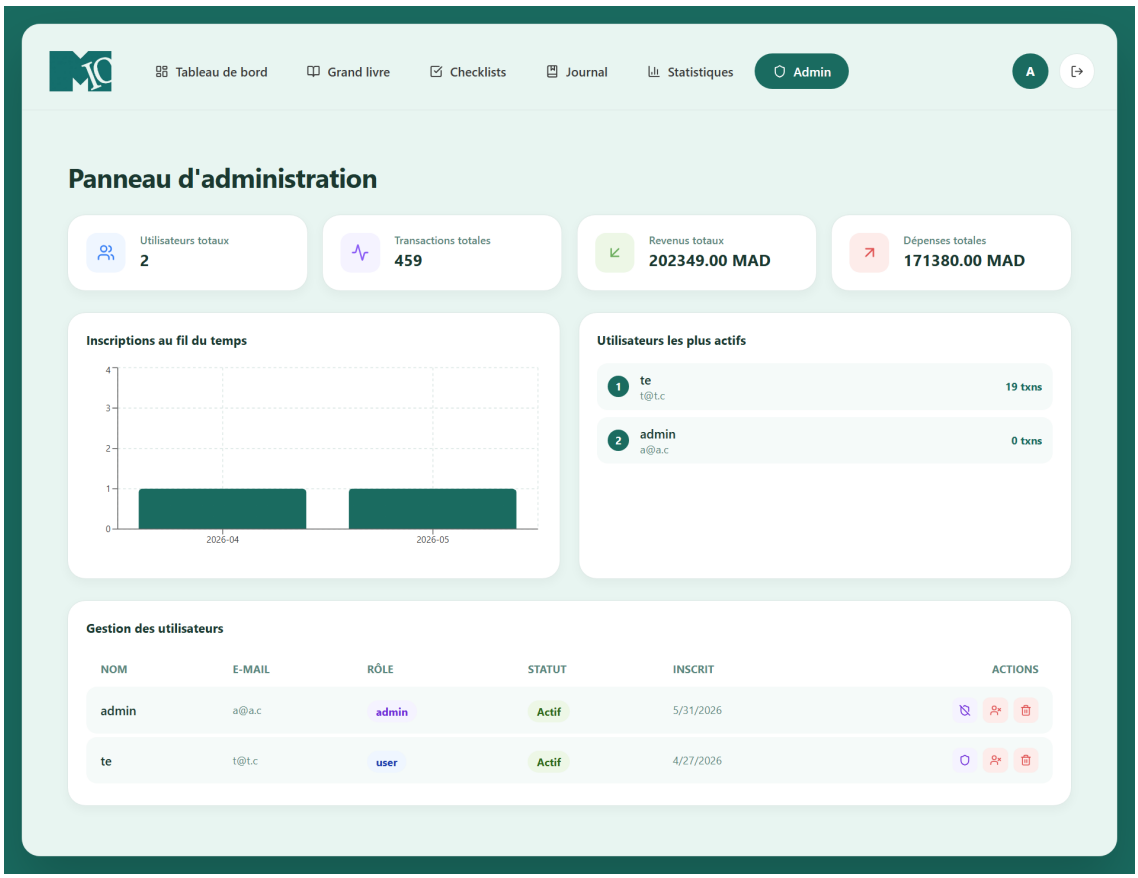


FIGURE 4.32 – Panneau d'administration

Le panneau d'administration se compose de trois zones principales :

- **Indicateurs globaux :** Quatre cartes de synthèse affichent en temps réel le nombre total d'utilisateurs inscrits, le nombre total de transactions enregistrées sur la plateforme, ainsi que le cumul des revenus et des dépenses tous utilisateurs confondus.
- **Statistiques système :** Un graphique en histogramme (*Inscriptions au fil du temps*) retrace l'évolution du nombre d'inscriptions par mois, permettant de suivre la croissance de la plateforme. Un classement des *Utilisateurs les plus actifs* met en avant les comptes ayant enregistré le plus grand nombre de transactions.
- **Gestion des utilisateurs :** Un tableau recense l'ensemble des comptes de la plateforme (nom, e-mail, rôle, statut, date d'inscription) et propose trois actions par utilisateur :
  - **Modifier le rôle** (icône bouclier) : bascule le compte entre les rôles *user* et *admin* ;
  - **Désactiver le compte** (icône utilisateur barré) : suspend l'accès de l'utilisateur à la plateforme sans supprimer ses données ;

— **Supprimer le compte** (icône corbeille) : supprime définitivement l'utilisateur et les données associées.

Ces fonctionnalités confirment la couverture des principaux cas d'utilisation attribués à l'acteur Administrateur lors de la phase de conception, à l'exception de la gestion de paramètres globaux à l'échelle de l'application (au-delà des préférences propres à chaque utilisateur), qui n'a pas été implémentée dans le cadre de ce stage et constitue une piste d'amélioration future.

4.16 Conclusion

En somme, ce quatrième chapitre a permis de valider la parfaite adéquation entre les exigences spécifiées lors de la phase de conception et le produit logiciel final. L'alliance d'une interface utilisateur moderne, bâtie sur React et Tailwind, et de modules innovants comme l'IA financière et le journal contextuel Nikki, dote la plateforme MoneyIO d'une réelle valeur ajoutée en matière d'expérience utilisateur. Les tests fonctionnels menés à travers ces différents scénarios d'utilisation confirment la stabilité, la rapidité et la robustesse de l'application.

Conclusion Générale

Ce stage de fin de formation a constitué une expérience particulièrement enrichissante, tant sur le plan technique que sur le plan professionnel. Il m'a permis de mettre en pratique les connaissances acquises durant ma formation tout en découvrant les différentes étapes de conception et de développement d'une application web complète.

Le projet réalisé, **MoneyIO**, est une application web de gestion des finances personnelles permettant aux utilisateurs de suivre leurs revenus, leurs dépenses, leurs comptes, leurs budgets, leurs notes et leur journal personnel au sein d'une interface moderne et intuitive. Afin d'apporter une valeur ajoutée à l'application, un module d'intelligence artificielle a également été intégré pour fournir des analyses financières personnalisées et des recommandations adaptées aux habitudes de dépenses des utilisateurs.

Le développement de cette application s'est appuyé sur une architecture moderne reposant sur React pour le frontend et Laravel pour le backend, communiquant via des API REST sécurisées grâce à Laravel Sanctum. Cette approche a permis de concevoir une application modulaire, performante et facilement évolutive.

Le contexte du stage, limité dans le temps, a naturellement contraint le périmètre fonctionnel final de l'application. Plusieurs fonctionnalités envisagées lors de la phase de conception n'ont ainsi pas pu être implémentées, notamment la liaison directe d'une carte bancaire pour synchroniser automatiquement les transactions, la création de comptes enfants permettant une gestion budgétaire familiale, un gabarit budgétaire automatisé basé sur la règle des 50/30/20 (besoins essentiels, envies, épargne), une catégorie dédiée au suivi des dons (Sadaqa), ainsi que la distinction entre revenus fixes (salaire mensuel) et revenus variables (freelance, primes, revenus ponctuels).

Au-delà des aspects techniques, ce projet m'a permis de développer des compétences essentielles telles que l'autonomie, la gestion du temps, l'organisation d'un projet complet, la résolution de problèmes ainsi que la recherche de solutions adaptées face aux difficultés rencontrées.

Ces limites ouvrent naturellement la voie à plusieurs perspectives d'évolution pour les futures versions de l'application : l'intégration de la liaison de cartes bancaires et de la synchronisation automatique des transactions, le développement d'un système de comptes enfants, l'ajout de gabarits budgétaires reconnus (dont la règle des 50/30/20), la création d'une catégorie dédiée aux

dons, la distinction entre revenus fixes et variables pour affiner les analyses de l'IA financière, le développement d'une version mobile native, ainsi que l'intégration d'un système de notifications intelligentes basé sur les habitudes de dépenses de l'utilisateur.

Ce stage représente ainsi une étape importante dans mon parcours de formation et constitue une expérience concrète qui renforce mes compétences en développement web full-stack ainsi que ma capacité à mener un projet informatique de sa conception jusqu'à sa réalisation.